Q-DECODEQEC

Comparative Decoding of Real IBM Quantum Repetition-Code Syndromes and a Passive versus Active Memory-Correction Experiment on ibm_fez

COMPLETE TECHNICAL REPORT WITH FIGURES AND OUTPUT DATA

Trinexis × IEEE | IBM Qiskit Fall Fest 2026

Track: Quantum Error Correction and Fault Tolerance

Prepared by

Kaustubh Ladda | GitHub: kaustubhkladda

Evidence snapshot: notebook main.ipynb and outputs/ directory, generated 26 October 2026 (UTC)

Scope of this report

Every number in this report is taken from the supplied `main.ipynb` outputs and the files in `outputs.zip` (10 figures, 6 JSON files, 1 JSONL ledger, 6 prediction CSVs, 3 analysis CSVs, 1 raw-shot CSV, QPY circuit files, frozen model files and `findings.txt`). Where a number was derived directly from a supplied CSV (for example shot counts per register value), the text says so.

The final-test set has no labels, so no accuracy or logical-error rate is reported for it.

Contents

Section	Title
1	Abstract
2	Executive summary
3	Introduction and problem formulation
4	Objectives and scope
5	Quantum background and metrics
6	Environment, configuration and reproducibility settings
7	Data provenance and integrity verification
8	Schema audit and detector-layout verification
9	Exploratory data analysis
10	Feature register and leakage prevention
11	Matching graph and M0 decoder
12	M1 neural decoder
13	M2 difficulty-aware router
14	Internal-test benchmark
15	Probability calibration
16	Model freeze and final predictions
17	Real-hardware setup and budget guard
18	Circuit design, transpilation and diagrams
19	Calibration job and Part A hardware results
20	Frozen decoders on live passive shots
21	Job recovery, verification and QPU ledger
22	Single-syndrome demonstration
23	Latency and computational cost
24	Findings, scorecard and end-goal comparison
25	Security, simulator and secret audits
26	Interpretation and discussion
27	Threats to validity and limitations
28	Conclusion and future work
A	Appendix A: complete JSON output files
B	Appendix B: complete CSV analysis outputs and data samples
C	Appendix C: evidence index (34 files)
D	Appendix D: equations and definitions
E	Appendix E: references

1. Abstract

This project investigates practical decoding for a distance-3, three-round bit-flip repetition-code memory using shot records from IBM Quantum hardware. For each shot, an eight-bit detector vector is given and the task is to predict whether the logical observable flipped. Three required decoders are compared: **M0**, a PyMatching minimum-weight perfect matching (MWPM) decoder; **M1**, a classical multilayer perceptron (MLP) with a validation-tuned decision threshold; and **M2**, a difficulty-aware router that selects an identity, matching or neural path per shot and calibrates neural probabilities with Platt scaling. A no-flip baseline and a detector-pattern lookup table are included as controls so that weak comparisons cannot look stronger than they are.

The labelled data (4,000 training and 1,000 internal-test shots, originally measured on `ibm_marrakesh`) are split by circuit; a further 2,000 shots are unlabelled and reserved for final predictions. On the 1,000-shot internal test the logical error rate (LER) is 0.245 for the no-flip baseline, 0.020 for the lookup control, 0.028 for M0, 0.018 for M1 and 0.020 for M2. Paired bootstrap intervals show that decoding is clearly better than predicting no flip, but they do not resolve the differences among the lookup control, M0, M1 and M2. Applying the predeclared tie-break (lowest median single-shot latency), M0 is the exploratory recommendation at 30.3 µs per shot.

Separately, a set of 16 passive/active memory circuits (8 fault configurations × 2 versions, 250 shots each) was executed in one job on the real `ibm_fez` backend. Across 2,000 passive shots the raw LER was 0.2535; offline M0 decoding reduced it to 0.0180. The active circuit, which applies a conditional X correction on the device based on its last syndrome register, went from 0.2530 before feedback to 0.0480 after correction, a large improvement but still less effective than offline decoding. The work provides a reproducible, hash-verified real-hardware experiment; it does not establish quantum advantage or general fault tolerance.

Keywords: quantum error correction, repetition code, syndrome decoding, PyMatching, minimum-weight perfect matching, multilayer perceptron, calibration, dynamic circuits, IBM Quantum, logical error rate.

2. Executive summary

> **Main result**
> Decoding reduced internal-test LER from 24.5% (no-flip baseline) to 1.8% (M1 MLP). The advantage over the pattern-lookup control (2.0%) is small and is not statistically resolved by the paired interval, so the result is promising but not conclusive.

Item	Measured result	Meaning
Labelled data	4,000 train; 1,000 internal test	All labelled rows use the same eight detector-bit feature definition.
Unlabelled holdout	2,000 final-test shots	Predictions created after model freeze; no score can be computed.
Best internal-test point estimate	M1 MLP: LER 0.018; accuracy 0.982	Lowest LER of the three required decoders; not proven better than lookup or matching.
Selection-rule recommendation	M0 MWPM; p50 30.3 µs/shot	Exploratory choice after the paired-interval tie rule and then the latency tie-break.
Real-device active correction	LER 0.2530 before feedback; 0.0480 after	Reduction of 20.5 percentage points; residual error expected because correction uses the last syndrome only.

Item	Measured result	Meaning
Offline decoding on live passive shots	M0 LER 0.0180	Best point estimate in the 2,000-shot passive sample; most comparisons with M0 are not resolved.
QPU usage	2.0 s calibration + 3.0 s Part A = 5.0 s	Recorded by completed jobs re-queried from IBM and verified as real-device jobs.
Integrity and security	22 data files verified; 34 evidence files indexed; secret scan PASS	Hashes, a model-freeze manifest and a job ledger support traceability.

2.1 Key contributions

- Built and evaluated three decoding strategies against both a trivial baseline and a strong pattern-lookup control on identical labelled shots.
- Constructed a PyMatching graph (8 detector nodes, 18 edges, 1 logical-fault identifier) and verified it against eight hand-made syndrome cases (8/8 pass).
- Used a fixed seed (2026), train-only hyperparameter search, a frozen model manifest with SHA-256 hashes, validated prediction files, paired bootstrap intervals and explicit leakage guards.
- Compiled and executed passive and active repetition-code memory circuits on `ibm_fez` using mid-circuit measurements, resets and classical conditional operations.
- Reported negative and inconclusive results openly: statistical ties, M2 latency overhead, and the limits of single-syndrome feedback.

3. Introduction and problem formulation

3.1 Motivation

Physical qubits are imperfect: bit-flip errors and measurement faults can corrupt quantum information. Quantum error correction encodes logical information across several physical qubits and extracts syndrome information that signals error patterns without directly measuring the encoded state. A decoder maps syndrome-derived features to a correction decision. For small codes, a decoder should be judged not only on prediction quality but also on latency, calibration, reproducibility and behaviour on real hardware.

This project uses a three-data-qubit repetition code with distance three and three rounds of syndrome extraction. The code targets bit-flip (X) errors, not arbitrary quantum noise. The experiment compares offline decoding from measured detectors with a separate active memory circuit that applies a conditional X correction inside the circuit based on its most recent syndrome register. These are different control paths and are analysed separately.

3.2 Formal task

For each shot i the input is a binary detector vector x_i in {0,1}^8. The target y_i in {0,1} is `observable_error`, which equals 1 when the designated logical observable differs from its prepared value. Each decoder returns a predicted flip ŷ_i. The main metric is the logical error rate LER = (1/N) Σ 1[ŷ_i ≠ y_i]. The task is deliberately narrow: it does not predict arbitrary Pauli errors, does not generalise automatically to other code distances, and does not establish fault tolerance at scale.

3.3 Source data

The labelled source data are real IBM Quantum records from `ibm_marrakesh` (job `dbbq3agc77vs7384te2g`, completed 2026-10-20 17:32 UTC): eight passive circuit configurations

with 625 shots each (5,000 records), split by circuit name with split seed 20261020 into 4,000 training and 1,000 internal-test rows. The dataset card states `simulated_data: false`. A later 2,000-row unlabelled table is reserved for final predictions. The active/passive experiment of this project is a distinct experiment executed on `ibm_fez`; the two backends must not be conflated.

4. Objectives and scope

4.1 Objectives

- Implement a matching-graph decoder (M0), a neural detector classifier (M1) and a multi-option routing decoder (M2).
- Benchmark methods on the same internal-test shots with bootstrap confidence intervals and paired comparisons.
- Keep the final unlabelled set out of model selection; produce validated prediction CSVs from frozen models.
- Build passive and active memory circuits, transpile them for a real IBM backend, and compare measured logical error rates.
- Record hardware job identifiers and QPU usage; verify jobs, shot counts, data files, model hashes and secret handling.

4.2 In scope and out of scope

In scope	Out of scope / not claimed
Distance-3 bit-flip repetition code; three rounds of two-check syndrome measurement.	Surface-code-scale decoding or logical qubits at production scale.
Classical CPU decoding from eight binary detector bits.	A quantum machine-learning model executed on the QPU.
Real IBM hardware experiment with passive versus active correction.	Demonstration of asymptotic fault tolerance or quantum advantage.
M0, M1, M2; lookup and no-flip controls; S1/S2 marked NOT_ATTEMPTED.	QSVC or QAOA+VQC performance numbers.
2,000 final-test predictions with quality checks.	A final-test LER or accuracy (labels not supplied).

5. Quantum background and metrics

5.1 Distance-3 repetition code

The repetition code stores one logical bit across three data qubits. Under ideal bit-flip-only noise, majority vote recovers the logical bit, but real syndrome extraction adds measurement faults and timing effects. Two ancilla qubits measure the two adjacent parity checks in each of three rounds; repeated measurement reveals changes between rounds so that detector events capture temporal inconsistencies. The code has distance three: the idealised code corrects one bit flip, and two or more physical errors can cause a logical failure.

5.2 Syndrome bits and detector bits

Each round returns two syndrome bits, giving six stored syndrome bits. The eight detector bits are derived from the syndrome history and final data readout according to the organiser convention (indexed `detectors[2*t+i]`, with t = 0..3 time slices and i = 0..1 checks). The notebook reconstructs these detector bits from the source columns and obtains exact agreement on all 5,000 labelled rows; the same parser is applied to live hardware registers. Model inputs are the eight detector bits only.

5.3 Metrics

LER is the fraction of incorrectly decoded shots and accuracy equals 1 − LER. Precision, recall and F1 describe the positive (logical flip) class. Because 24.5% of internal-test shots are positive, a no-flip classifier already reaches 75.5% accuracy, so accuracy alone is not a sufficient success criterion. Single-method confidence intervals use bootstrap resampling (1,000 resamples); paired differences use the same resampled indices for both methods (2,000 paired resamples). If a 95% paired interval contains zero the notebook labels the comparison `NOT_STATISTICALLY_RESOLVABLE`. Wilson intervals are used for single proportions in the hardware section.

6. Environment, configuration and reproducibility settings

6.1 Runtime environment

The notebook ran on Python 3.12.10 under Windows 11 (build 10.0.26300). Every installed package version matched the tested version (0 packages differ).

Package	Tested	Installed	Status
qiskit	2.5.0	2.5.0	same
qiskit_ibm_runtime	0.48.0	0.48.0	same
pymatching	2.4.0	2.4.0	same
pandas	2.3.2	2.3.2	same
numpy	2.1.0	2.1.0	same
scipy	1.18.0	1.18.0	same
sklearn	1.7.1	1.7.1	same
matplotlib	3.10.6	3.10.6	same
joblib	1.5.3	1.5.3	same

6.2 Experiment configuration (outputs/config.json)

All settings live in a single configuration object, saved to `outputs/config.json`. Seed 2026 drives every split, bootstrap and optimiser. Credentials were present (token, instance CRN, channel `ibm_quantum_platform`) and are read from environment variables or a git-ignored `.env` file; they are never printed.

Parameter	Value	Role
seed	2026	Random seed for splits, bootstrap and optimiser
boot_single / boot_paired	1000 / 2000	Bootstrap resamples for single metrics / paired differences
cv_folds	5	Stratified cross-validation folds
matching_ratio_grid	0.5, 0.75, 1.0, 1.25, 1.5, 2.0, 3.0	Measurement/data edge-weight ratios searched for M0
mlp_sizes	(16), (32,32), (64,64), (128,64,32)	Hidden-layer structures searched for M1
mlp_alphas	0.0001, 0.01	L2 weight decay values
mlp_max_epochs / patience / batch / lr	150 / 15 / 64 / 0.001	Training controls
slices (M2)	fit 0.6, val 0.2, route 0.2	Disjoint M2 training slices
difficulty	trivial w=0; easy w in {1,2}; hard w>=3	Detector-weight difficulty classes

Parameter	Value	Role
shots_partA / shots_calibration	250 / 100	Shots per circuit on hardware
qpu_budget_s / reserve / partA cap / calibration cap	600 / 130 / 40 / 30 s	QPU-time guard
transpile_optimization_level	1	Qiskit transpiler level
submit_to_qpu	true	Hardware submission enabled
latency_single_shots / batch_repeats	300 / 15	Latency benchmark sizes

The complete raw `config.json` is reproduced in Appendix A.

7. Data provenance and integrity verification

7.1 Dataset card

The organiser-supplied dataset card records the origin of the labelled data.

Field	Value
backend	ibm_marrakesh
job_id	dbbq3agc77vs7384te2g
source	IBM Quantum physical backend
simulated_data	false
submitted_at_utc	2026-10-20T17:15:23.048113+00:00
completed_at_utc	2026-10-20T17:32:04.729349+00:00
layout (physical qubits)	110, 111, 98, 91, 92
shots_per_circuit	625
rows	full 5000 / train 4000 / test 1000
split	stratified by circuit_name; split_seed 20261020

The eight source circuits are `rep_d3_r3_b0_passive_natural`, `rep_d3_r3_b1_passive_natural`, and six single-X-fault circuits `rep_d3_r3_b0_passive_xq{0,1,2}_after{0,1}`. The IBM result-verification file reports `shot_aligned_registers_match_ibm: true` and 625 shots checked for each of the eight circuits.

7.2 Pre-submission calibration snapshot of the source qubits

The source manifest stores calibration data captured at 2026-10-20 17:15:22 UTC (backend last update 2026-10-20 12:01:20-05:00) for the five qubits used on `ibm_marrakesh`.

Qubit	Readout error	T1 (µs)	T2 (µs)
110	0.012207	290.8	241.2
111	0.016602	130.7	87.8
91	0.006958	121.9	175.3
92	0.007080	174.0	120.0
98	0.006348	108.3	54.2

Two-qubit gate (CZ)	Error
cz:110,111 and cz:111,110	0.001882
cz:111,98 and cz:98,111	0.002029

Two-qubit gate (CZ)	Error
cz:91,92 and cz:92,91	0.003002
cz:91,98 and cz:98,91	0.002109

In the source manifest, the natural logical-0 circuit has depth 37 and the other circuits listed there have depth 38; each has 12 CZ gates, 9 measurements and 6 resets, and the logical-1 natural circuit and X-injection circuits add X gates (3 and 1 respectively).

7.3 SHA-256 verification of data files

The notebook verified 22 data files: 19 by raw-byte SHA-256 and 3 JSON files by normalised-JSON SHA-256. Raw bytes of the three JSON files differ from the manifest expectation (for example `dataset_card.json`: 839 original bytes vs 807 normalised bytes; likely line-ending differences), but the normalised-JSON comparison matches the expected hashes. All 22 checks are marked OK.

File	Expected = computed SHA-256	Method
dataset_card.json	f1f13a49a161e474225846f524e58714401e3d45b039fe2d447bf452c4ca5ff5	normalized-json
final_test_inputs.parquet	99e219cc34f748434dd1b70cc78c5b4270d1d0d43f6937464644f04328831998	raw
internal_test_1000.parquet	02c61b3e5d9cd71fd0a6c7f1c15cb022f4325d8b62112e164c37a7606fda2f51	raw
raw/…/ibm_result_verification.json	a15866fb0309a216ed9bf9b1eba3792ac6a9e24e3a5367549307d0f3da389577	normalized-json
raw/…/manifest.json	d0c8052620457305a8fda35ada504feaec516b6410c7921ab210ca067efa6870	normalized-json
raw/…/rep_d3_r3_b0_passive_natural.npz	2f58439ff51c82d2e554e32d29b321efaeaeaf06cd4c99239140845ddd14efb3	raw
raw/…/rep_d3_r3_b0_passive_natural.qpy	01c18f60551cc852e419596a477c6769ca4e1cf7cb9d5a89fb0fa2b455af8c64	raw
raw/…/rep_d3_r3_b0_passive_xq0_after0.npz	ee478715bd54c8e7e4cb0cbf2660d7d84cc041e6cf5dddf959399df81f849b37	raw
raw/…/rep_d3_r3_b0_passive_xq0_after0.qpy	df711e5ba62eaa115fcca26e93c884a8395ebdca06347f456bce9fb585429839	raw
raw/…/rep_d3_r3_b0_passive_xq0_after1.npz	5bbfcc1b53e90e20b7e0e57010b18b2a0ae6a1ed9acac554e582e0342c2de1c9	raw
raw/…/rep_d3_r3_b0_passive_xq0_after1.qpy	dbc2aacf898d8e4daf74bfd72f54eeb7cb347934b0148475e8dedd1465dd7eae	raw
raw/…/rep_d3_r3_b0_passive_xq1_after0.npz	8be543d355e7068f800d639dbd7cb0767531337eef643ff161ec6a92117e7ca1	raw
raw/…/rep_d3_r3_b0_passive_xq1_after0.qpy	9101cfb3dc777b9967144138a60ae56406b26401215ad888ab13bf501c7bef35	raw
raw/…/rep_d3_r3_b0_passive_xq1_after1.npz	f1d455aec9fbeaa39d4974a52ed0b2955b625078b72ccdc61060ff971d354fe4	raw
raw/…/rep_d3_r3_b0_passive_xq1_after1.qpy	2057d7d6101ba1553425e9cc9248cb4c1b0b3ada574dc352a28d5243fdf2d806	raw
raw/…/rep_d3_r3_b0_passive_xq2_after0.npz	c95a055a0a8a4172c7bc1ffadbfdaf22325a8520363d85ea2dc4d8aebd29b4d9	raw
raw/…/rep_d3_r3_b0_passive_xq2_after0.qpy	323816dbd70cef6a1c73bb6f8ca13e644c7c11355084b1afac56448106140afb	raw
raw/…/rep_d3_r3_b0_passive_xq2_after1.npz	7bf0668f0a6b7c2bf968c1ffd60848790aacba1be5443e3b068a70a6ead539a8	raw
raw/…/rep_d3_r3_b0_passive_xq2_after1.qpy	82b49194aacc36fa1375e6031df345e74497c6e8fd72f9db9782a01a2ec507b3	raw
raw/…/rep_d3_r3_b1_passive_natural.npz	515d65d97087d9b925dc44d8d8afd9c117437a38ff34be864f84a96d699566ce	raw

File	Expected = computed SHA-256	Method
raw/…/rep_d3_r3_b1_passive_natural.qpy	d93f42e8b197d9b946b95369f7a650f0f27256bdfa32a009170dc963a6e9c9c4	raw
train_4000.parquet	203790f8c619e0a09ade1bc7ab35e650119d8224354f6ac30c477d97856d881d	raw

JSON file	Original SHA-256	Normalised SHA-256	Bytes (orig / norm)
dataset_card.json	6a79e75fd5173877…	f1f13a49a161e474…	839 / 807
ibm_result_verification.json	1ede5d56e38e40a6…	a15866fb0309a216…	554 / 539
raw manifest.json	b3d8859245ba9f50…	d0c8052620457305…	10158 / 9793

8. Schema audit and detector-layout verification

8.1 Table audit

Split	Rows	Columns	Null cells	Detector shape	Syndrome shape	Values	Duplicate IDs
train	4,000	18	0	(4000, 8)	(4000, 6)	[0, 1]	0
internal_test	1,000	18	0	(1000, 8)	(1000, 6)	[0, 1]	0
final_test	2,000	3	0	(2000, 8)	(2000, 6)	[0, 1]	0

Training and internal-test columns: `record_id`, `shot_id`, `circuit_name`, `job_id`, `backend`, `distance`, `rounds`, `initial_bit`, `fault_type`, `injected_x_qubit`, `injected_after_round`, `fault_label`, `syndrome`, `detectors`, `final_data`, `observable_error`, `detector_event_count`, `has_detection_event`. The final-test schema contains only `record_id`, `syndrome` and `detectors`, so it carries no target column.

8.2 Layout and label-definition checks

Six conventions were reconstructed from raw columns and compared with stored columns. All reach agreement 1.0.

Check	Agreement
train: detectors rebuilt from syndrome + final_data	1.0
train: observable_error == (final_data[0] != initial_bit)	1.0
internal_test: detectors rebuilt from syndrome + final_data	1.0
internal_test: observable_error == (final_data[0] != initial_bit)	1.0
bitstring little-endian round trip (real syndrome registers)	1.0
active-branch undo of commanded correction reproduces stored detectors	1.0

9. Exploratory data analysis

9.1 Class balance

Split	Shots	Flips	Flip rate	No-flip baseline accuracy
train	4,000	1,010	0.2525	0.7475
internal_test	1,000	245	0.2450	0.7550

9.2 Per-circuit label balance

Circuit	Train shots	Train flip rate	Test shots	Test flip rate
rep_d3_r3_b0_passive_natural	500	0.006	125	0.016
rep_d3_r3_b0_passive_xq0_after0	500	0.830	125	0.832
rep_d3_r3_b0_passive_xq0_after1	500	0.916	125	0.896
rep_d3_r3_b0_passive_xq1_after0	500	0.010	125	0.008
rep_d3_r3_b0_passive_xq1_after1	500	0.010	125	0.008
rep_d3_r3_b0_passive_xq2_after0	500	0.012	125	0.016
rep_d3_r3_b0_passive_xq2_after1	500	0.010	125	0.000
rep_d3_r3_b1_passive_natural	500	0.226	125	0.184

The flip rate is concentrated in circuits with an X fault on data qubit 0 (about 83% to 92% flips) and in the logical-1 natural circuit. Faults on data qubits 1 and 2 are almost always corrected by the code (flip rate near 1%), which is why the dataset is dominated by a few clear fault classes.

9.3 Detector weight and pattern coverage

Detector weight w	Train shots	Flip rate
0	803	0.000000
1	1,757	0.503130
2	1,142	0.032399
3	166	0.457831
4	120	0.075000
5	9	0.444444
6	3	0.000000

Split	Distinct patterns	Mean weight	Share zero weight	Pattern coverage by train	Share unseen in train
train	127	1.2705	0.20075	1.0000	0.0000
internal_test	68	1.2780	0.20200	0.9950	0.0050
final_test	94	1.2635	0.19950	0.9915	0.0085

Only 127 distinct eight-bit patterns occur in training, about 31 shots per pattern on average, so an exact lookup table is a strong control and a neural network has limited room to add information. The final-test unseen share (0.85%) is a descriptive drift indicator only.



Figure 1a. Class balance (train)

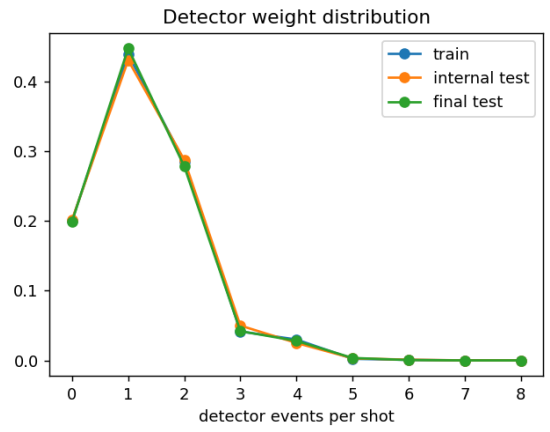


Figure 1b. Detector-weight distribution by split

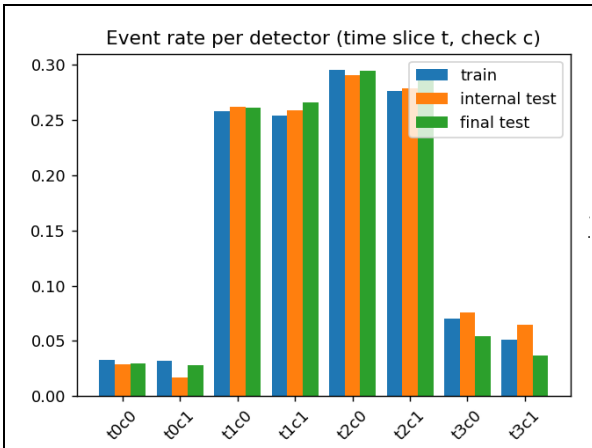


Figure 1c. Event rate per detector (time slice t, check c)

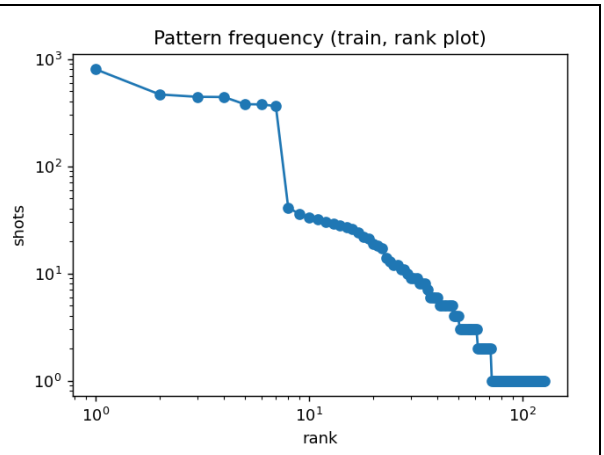


Figure 1d. Pattern frequency rank plot (train, log-log)

Figure 1 (fig_eda.png, shown as four panels). Panel 1a shows 74.8% no-flip against 25.2% flip. Panel 1b shows that train, internal-test and final-test detector-weight distributions almost coincide, peaking at weight 1. Panel 1c shows that rounds t1 and t2 carry most detector events (about 0.25 to 0.30 per detector) while t0 and t3 are quiet. Panel 1d shows seven very common patterns (roughly 360 to 800 shots each), then a steep drop to about 40 shots and a long tail of rare patterns.

10. Feature register and leakage prevention

Exactly one column, `detectors`, feeds the models. The function `guard_inputs` accepts a single eight-bit vector or an (n, 8) array, checks that all values are 0 or 1 and casts to an integer bit type. DataFrames and Series are rejected so that callers cannot pass full rows containing forbidden fields; the single-syndrome interface also rejects dictionaries and wrong vector widths. The guard rejected DataFrames, label columns, wrong width and non-binary values in tests.

Column	Group	Decision	Reason
detectors	query input	USED (M0, M1, M2)	8 detector events of one shot, the only model input
syndrome	query input	NOT USED (redundant)	equivalent 6-bit representation of the same history
record_id	identifier	excluded	joining and reporting only
shot_id	identifier	excluded	shot index inside its circuit, meaningless
job_id	provenance	excluded	constant, reporting only
backend	provenance	excluded	constant, reporting only
distance	provenance	excluded	constant (3)
rounds	provenance	excluded	constant (3)
observable_error	label	TARGET ONLY	training target and evaluation reference
final_data	label / audit	FORBIDDEN as input	determines the label exactly
initial_bit	label / audit	FORBIDDEN as input	determines the label exactly
fault_type	label / audit	FORBIDDEN as input	describes the injected cause
injected_x_qubit	label / audit	FORBIDDEN as input	describes the injected cause
injected_after_round	label / audit	FORBIDDEN as input	describes the injected cause
fault_label	label / audit	FORBIDDEN as input	describes the injected cause
circuit_name	label / audit	stratification + EDA only	encodes the injected fault; used only to stratify
detector_event_count	label / audit	FORBIDDEN as input	derived from the input, EDA only
has_detection_event	label / audit	FORBIDDEN as input	derived from the input, EDA only

- Models are frozen and hashed before the final-test predictions are generated.

- Prediction files are validated for row count, record-ID set, allowed prediction values and probability range.
- Detector layout and label definition match on all 5,000 labelled source rows.

11. Matching graph and M0 decoder

11.1 Controls

The **no-flip control** always predicts 0: 245 mistakes in 1,000 shots (LER 0.245, recall 0). The **pattern lookup** maps each training pattern to its majority label (ties go to 0; unseen patterns default to no flip). With 127 training patterns and 99.5% of internal-test shots seen, it reaches LER 0.020.

11.2 Graph unit tests

The PyMatching graph has 8 detector nodes, 18 edges and one logical fault identifier. Data edges have weight 1; measurement and temporal edges carry a scalar ratio multiple. Eight hand-made syndromes verify the boundary interpretation; all pass.

#	Hand-made syndrome	Expected flip	Matching result	Status
0	no events	0	0	PASS
1	single event at n(0,0) (logical boundary)	1	1	PASS
2	single event at n(0,1) (other boundary)	0	0	PASS
3	single event at n(3,0)	1	1	PASS
4	pair n(0,0)+n(0,1): middle-qubit error	0	0	PASS
5	pair n(0,0)+n(1,0): measurement error	0	0	PASS
6	n(0,0)+n(2,1): two boundaries cheaper than a long path	1	1	PASS
7	n(1,1) alone	0	0	PASS

11.3 Edge-weight ratio selection (train-only, 5-fold CV)

Method	CV LER mean	CV LER sd
no-flip control (raw LER)	0.25250	0.008329
pattern-lookup control	0.01700	0.003021
MWPM ratio = 0.5	0.02075	0.005339
MWPM ratio = 0.75 (chosen)	0.02075	0.005339
MWPM ratio = 1.0	0.02875	0.004257
MWPM ratio = 1.25	0.02900	0.002669
MWPM ratio = 1.5	0.02900	0.002669
MWPM ratio = 2.0	0.02975	0.003102
MWPM ratio = 3.0	0.02975	0.003102

Ratios 0.5 and 0.75 tie at CV LER 0.02075; the declared tie rule chooses 0.75 as closer to 1.0. M0 then outputs the parity of the observable-carrying edges in the minimum-weight matching. PyMatching is a classical decoding library; no quantum computation is performed by the decoder itself [2].

12. M1 neural decoder

M1 maps eight detector bits to a probability of logical flip using a feed-forward network trained with binary cross-entropy and Adam (learning rate 0.001, batch 64), with balanced class weights

through resampling. Eight settings (four architectures × two L2 values) were compared by five-fold stratified CV; each fold was early-stopped on a held-out 20% slice (patience 15, maximum 150 epochs) and the decision threshold was tuned on validation data rather than fixed at 0.5.

12.1 Architecture and weight-decay search

Hidden layers	Alpha	Class weight	Params	CV LER (tuned)	CV sd	CV LER (thr 0.5)	Mean thr.	Mean epochs
(16)	0.0001	True	161	0.02250	0.006673	0.03750	0.724	149.4
(16)	0.0100	True	161	0.02175	0.006034	0.04150	0.696	149.0
(32, 32)	0.0001	True	1,377	0.02125	0.006435	0.02625	0.796	61.2
(32, 32)	0.0100	True	1,377	0.01975	0.006212	0.02400	0.812	77.4
(64, 64)	0.0001	True	4,801	0.01850	0.003236	0.02450	0.784	47.4
(64, 64)	**0.0100**	True	4,801	**0.01750**	0.003644	0.02600	0.814	50.4
(128, 64, 32)	0.0001	True	11,521	0.02050	0.004384	0.02900	0.662	32.0
(128, 64, 32)	0.0100	True	11,521	0.01950	0.003812	0.02675	0.772	34.2

The chosen setting (64, 64), alpha 0.01 has the lowest CV LER (ties resolved to the smaller network). The spread over all eight settings is only 0.005, comparable to the fold standard deviation of about 0.005, so the architecture choice matters little. Search time was 42 s.

12.2 Ablation

Variant	CV LER
chosen: class weights + tuned threshold	0.01750
class weights + threshold 0.5	0.02600
no class weights + tuned threshold	0.01775
no class weights + threshold 0.5	0.01850
network size (16), best weight decay	0.02175
network size (32, 32), best weight decay	0.01975
network size (64, 64), best weight decay	0.01750
network size (128, 64, 32), best weight decay	0.01950

Threshold tuning had the largest measurable effect (0.0175 versus 0.0260 at a fixed threshold of 0.5). Removing class weights while keeping the tuned threshold changed CV LER only from 0.01750 to 0.01775.

12.3 Final M1 record

Item	Value
Configuration	hidden (64, 64), alpha 0.01, class_weight true
Parameter count	4,801
Epochs before stopping	58
Fit / validation rows	3,200 / 800
Validation log-loss	0.102852
Validation threshold	0.84
Validation LER	0.01375
Mean cross-validated LER	0.01750

13. M2 difficulty-aware router

M2 wraps classical candidate decoders in a five-stage routing policy. The detector Hamming weight w defines difficulty classes: TRIVIAL (w = 0), EASY (w = 1 or 2) and HARD (w ≥ 3). Training data are divided into three disjoint slices (disjointness is asserted in code).

Slice	Shots	Flip rate	Use
fit	2,400	0.245833	Select matching ratio; fit the neural pre-decoder
validation	800	0.262500	Early stopping, calibration, threshold
routing	800	0.262500	Compare candidate error counts per difficulty class

Difficulty	Routing shots	IDENTITY errors	MWPM errors	NEURAL errors	Chosen
TRIVIAL	155	0	0	0	IDENTITY
EASY	593	192	5	4	NEURAL
HARD	52	18	13	12	NEURAL

The routing table is TRIVIAL → IDENTITY, EASY → NEURAL, HARD → NEURAL. The neural probability is calibrated by Platt scaling with a = 0.8385 and b = −0.9934 (reported as 0.838 and −0.993) and a final threshold of 0.50. Expected calibration error (ECE) on the routing slice fell from 0.023 to 0.009. A matching cache reproduces non-cached matching predictions exactly on 5,000 shots with a 99.0% hit rate. The router avoids a neural call on the 20.2% of internal-test shots with zero detector weight.

13.1 Optional stretch methods

S1 (quantum-kernel SVM) and S2 (QAOA + variational quantum classifier) are marked NOT_ATTEMPTED. No QPU measurement or simulation exists for them and the single-syndrome interface returns that status explicitly. The report makes no claim about their performance.

14. Internal-test benchmark

14.1 Selection rule (declared before scoring)

Eligible models must be frozen, use only the eight detector bits and avoid leakage. The lowest LER is considered first; candidates whose paired 95% interval against the best contains zero form a tie group, and the lower batch-one median latency breaks the tie. If the best method cannot be distinguished from the lookup control, the result is labelled inconclusive and the recommendation is exploratory.

14.2 Metrics on 1,000 identical shots

Method	LER [95% CI]	Accuracy [95% CI]	Balanced acc. [95% CI]	Precision	Recall	F1 [95% CI]	TN/FP/FN/TP
No-flip control	0.245 [0.215, 0.273]	0.755 [0.727, 0.785]	0.500 [0.500, 0.500]	n/a	0.000	n/a	755/0/245/0
Pattern lookup	0.020 [0.012, 0.029]	0.980 [0.971, 0.988]	0.969 [0.953, 0.982]	0.971	0.947	0.959 [0.939, 0.976]	748/7/13/232
M0 MWPM	0.028 [0.018, 0.038]	0.972 [0.962, 0.982]	0.968 [0.954, 0.980]	0.929	0.959	0.944 [0.922, 0.963]	737/18/10/235
M1 MLP	0.018 [0.010, 0.027]	0.982 [0.973, 0.990]	0.970 [0.955, 0.984]	0.979	0.947	0.963 [0.944, 0.979]	750/5/13/232
M2 MOR router	0.020 [0.012, 0.029]	0.980 [0.971, 0.988]	0.970 [0.956, 0.984]	0.967	0.951	0.959 [0.939, 0.976]	747/8/12/233

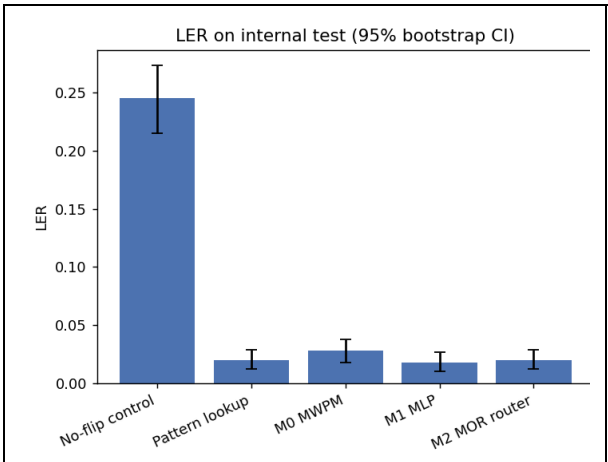


Figure 2a. LER on internal test (95% bootstrap CI)

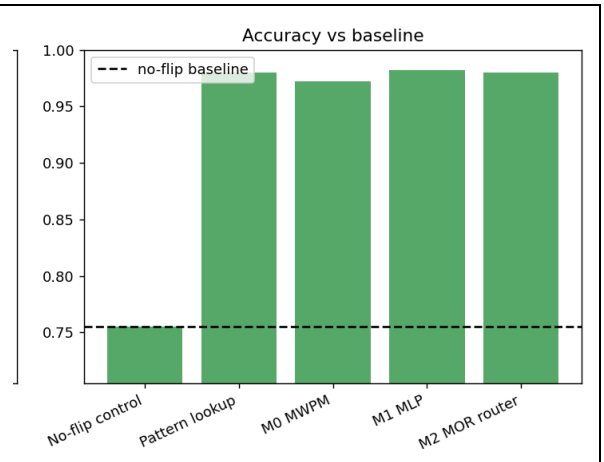


Figure 2b. Accuracy versus no-flip baseline

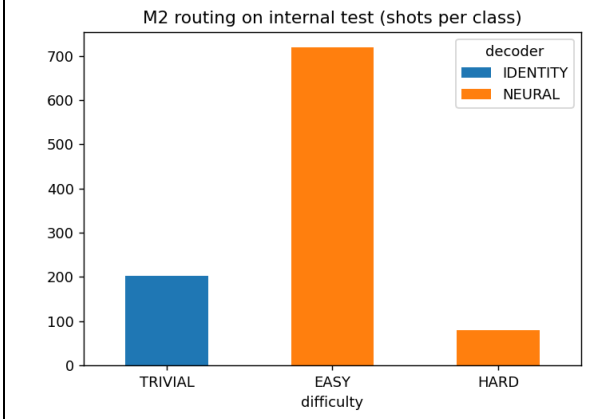


Figure 2c. M2 routing on internal test (shots per class)

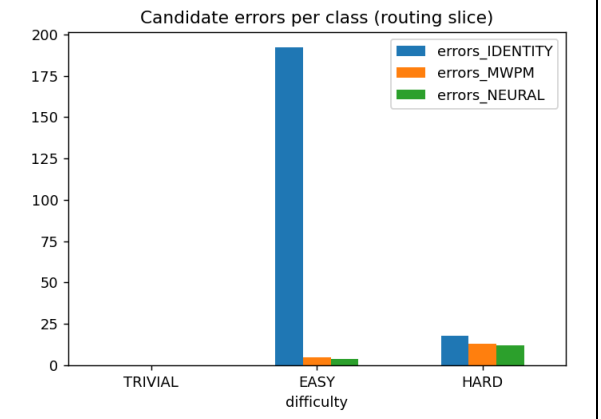


Figure 2d. Candidate errors per class (routing slice)

Figure 2 (fig_benchmark.png, shown as four panels). Panel 2a shows all four decoders far below the no-flip LER of 0.245 with overlapping intervals. Panel 2b shows accuracy 0.972 to 0.982 against the 0.755 baseline. Panel 2c shows the final M2 routing of the internal test: 202 TRIVIAL shots to IDENTITY, 719 EASY and 79 HARD shots to NEURAL. Panel 2d shows that on the routing slice the identity path makes 192 errors on EASY shots while matching and neural paths make 5 and 4.

14.3 Paired comparisons

A	B	Delta LER	95% interval	Verdict
Pattern lookup	No-flip control	−0.225	[−0.252, −0.197]	A better (lower LER)
M0 MWPM	No-flip control	−0.217	[−0.246, −0.190]	A better (lower LER)
M1 MLP	No-flip control	−0.227	[−0.253, −0.200]	A better (lower LER)
M2 MOR router	No-flip control	−0.225	[−0.252, −0.197]	A better (lower LER)
M1 MLP	M0 MWPM	−0.010	[−0.021, 0.000]	NOT_STATISTICALLY_RESOLVABLE
M2 MOR router	M0 MWPM	−0.008	[−0.019, 0.003]	NOT_STATISTICALLY_RESOLVABLE
M2 MOR router	M1 MLP	0.002	[−0.002, 0.006]	NOT_STATISTICALLY_RESOLVABLE
M0 MWPM	Pattern lookup	0.008	[−0.002, 0.019]	NOT_STATISTICALLY_RESOLVABLE
M1 MLP	Pattern lookup	−0.002	[−0.005, 0.000]	NOT_STATISTICALLY_RESOLVABLE
M2 MOR router	Pattern lookup	0.000	[−0.003, 0.003]	NOT_STATISTICALLY_RESOLVABLE

The benchmark establishes the value of decoding relative to predicting no flip, but not which non-baseline method is best. Even the numerically best comparison (M1 versus lookup) has an interval ending at zero and is classified as inconclusive.

14.4 Errors by difficulty class

Difficulty	Shots	True flips	No-flip errors	Lookup errors	M0 errors	M1 errors	M2 errors
TRIVIAL (0 detectors)	202	0	0	0	0	0	0
EASY (1–2 detectors)	719	227	227	8	9	8	8
HARD (3+ detectors)	79	18	18	12	19	10	12

14.5 Behaviour on patterns unseen in training

Method	Unseen shots	Errors on unseen	LER on unseen	Errors on seen
No-flip control	5	0	0.0	245
Pattern lookup	5	0	0.0	20
M0 MWPM	5	1	0.2	27
M1 MLP	5	0	0.0	18
M2 MOR router	5	0	0.0	20

Only five internal-test shots (0.5%) carry patterns absent from training, so this table carries almost no statistical weight.

15. Probability calibration

Calibration measurement	Value
M1 expected calibration error (internal test)	0.024
M2 raw neural probability ECE (internal test)	0.032
M2 calibrated probability ECE (internal test)	0.016
M2 ECE on routing slice before Platt scaling	0.023
M2 ECE on routing slice after Platt scaling	0.009

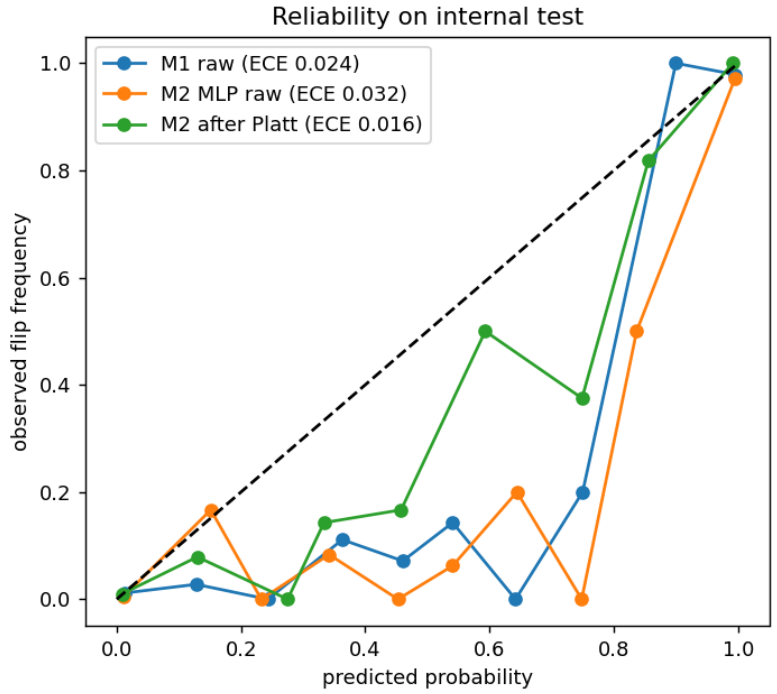


Figure 3. Reliability diagram on the internal test (fig_reliability.png): M1 raw (ECE 0.024), M2 MLP raw (ECE 0.032) and M2 after Platt scaling (ECE 0.016). The dashed diagonal is perfect calibration.

Platt scaling lowers ECE for M2 on both its routing slice and the internal test, Lower ECE does not by itself imply lower LER. No probability exists for M0, nor for M2 shots that follow the identity or matching path.

16. Model freeze and final predictions

16.1 Frozen artefacts

Five files were frozen and hashed at 2026-10-26 12:14:35.857901 UTC. The reloaded M1, M2 and lookup models reproduce in-memory internal-test predictions exactly (True, True, True) before any final prediction is generated.

Frozen file	SHA-256
m1.joblib	`e9f23ef66d058f0a9f895fdbe22edd980c77f238201700cdd1f957a00d26612f`
m2.joblib	`87723f087efe867c3501d9d90db4dbb0f4c76147588fff73cf5254fe50d7b854`
lookup.joblib	`94d454b42b8eef79af89cf26d1803aeb4058c7fd335b68d8b121a5520e17ee2e`
m0.json	`38def7a5c058d4480d949e31da1bdea243d1f16f8b8ac6a032e8af0d4d060757`
m2_slices_record_ids.json	`d7a82a0807dce7591c4cca0c64f713bbe0d1366da1cf05a2f850be6e17461381`

Frozen parameter	Value
seed	2026
M0 matching ratio	0.75
M1 configuration	hidden [64, 64], alpha 0.01, class_weight true
M1 threshold / epochs / parameters	0.84 / 58 / 4,801
M2 routing	TRIVIAL → IDENTITY; EASY → NEURAL; HARD → NEURAL
M2 MWPM ratio	0.75
M2 Platt (a, b)	0.8384677387653253, −0.9933551275259521
M2 neural threshold	0.5
M2 slice sizes	fit 2400, val 800, route 800

The full `frozen/manifest.json`, `frozen/m0.json` and `final_predictions_manifest.json` are reproduced in Appendix A.

16.2 Prediction files

File	Rows	Predicted flip rate	p_flip missing share	Valid
predictions_final_test_M0.csv	2,000	0.2600	1.0000	OK
predictions_final_test_M1.csv	2,000	0.2495	0.0000	OK
predictions_final_test_M2.csv	2,000	0.2530	0.1995	OK
predictions_internal_test_M0.csv	1,000	0.2530	1.0000	OK
predictions_internal_test_M1.csv	1,000	0.2370	0.0000	OK
predictions_internal_test_M2.csv	1,000	0.2410	0.2020	OK

Prediction file	SHA-256 (final_predictions_manifest.json)
predictions_final_test_M0.csv	`618122df07f32d2bb5e88c6aef9036766222f586d48de6b55c1c7503484a55ae`
predictions_final_test_M1.csv	`ffaf811b61fcb1d219e3bf877d78665a241ed80041d5fe9cab7dbe390e490c99`
predictions_final_test_M2.csv	`55d7b0d1b46cbf41191190d4129719665b34a36d0d7661a03ed88e0483264f3b`
predictions_internal_test_M0.csv	`0354bbe357fd65e8ca3dab44cbe27239c2060281ab2ef20a7821c66c0bf2da08`
predictions_internal_test_M1.csv	`c30a3c47589fc654e9a61495613f8b90714578cf041bea78806d072f1a12f3cb`

Prediction file	SHA-256 (final_predictions_manifest.json)
predictions_internal_test_M2.csv	5f0172a77cf4d6d2e47ccdb0fa877929d379216bc07ed780958f20ec56b16ecf

16.3 Descriptive statistics computed from the final-test CSVs

The following numbers were computed directly from the supplied CSV files (2,000 rows each). They are descriptive; there are no labels for the final test.

Quantity	M0	M1	M2
Predicted flips (count)	520	499	506
Predicted flip rate	0.2600	0.2495	0.2530
Rows with a p_flip value	0	2,000	1,601
p_flip mean	n/a	0.2742	0.3300
p_flip median	n/a	0.0105	0.0353
p_flip minimum	n/a	0.00022	0.00017
p_flip maximum	n/a	0.99947	0.99615

Pairwise agreement of final-test predictions	Share of 2,000 records
M0 and M1 agree	0.9765
M0 and M2 agree	0.9750
M1 and M2 agree	0.9965
All three agree	0.9740

M1 and M2 are almost identical on the final test, which is consistent with M2 routing most shots to the same neural network. M0 differs from the neural decoders on about 2.4% to 2.5% of records. The predicted flip rate of M2 moves from 0.241 (internal test) to 0.253 (final test) while mean detector weight moves from 1.278 to 1.2635; M2 skips the neural call on 20.2% (internal) and 19.95% (final) of shots.

Sample rows (first 8 rows of each final-test file)

record_id	M0 flip	M0 p_flip	M1 flip	M1 p_flip	M2 flip	M2 p_flip
0	0	(blank)	0	0.001897	0	0.004912
1	0	(blank)	0	0.001269	0	(blank)
2	0	(blank)	0	0.001897	0	0.004912
3	0	(blank)	0	0.053157	0	0.035256
4	0	(blank)	0	0.001269	0	(blank)
5	1	(blank)	1	0.999465	1	0.996152
6	0	(blank)	0	0.001269	0	(blank)
7	1	(blank)	1	0.999150	1	0.990538

17. Real-hardware setup and budget guard

17.1 Backend selection

Three candidate backends with dynamic-circuit support were listed. `ibm_fez` had zero pending jobs and was selected; it is a real device that supports `if_else` classical control. IBM documents this family of operations as dynamic circuits [3].

Backend	Qubits	Pending jobs	Dynamic circuits
ibm_fez (selected)	156	0	True

Backend	Qubits	Pending jobs	Dynamic circuits
ibm_kingston	156	35	True
ibm_marrakesh	156	1,998	True

17.2 Qubit-line scan

The notebook ranked 428 candidate five-qubit lines by combined readout and two-qubit error. The best five are listed below; rank 1 was chosen.

Rank	Line	Score	Readout error sum	Two-qubit error sum
1	(22, 23, 24, 25, 26)	0.041715	0.031250	0.010465
2	(143, 144, 145, 146, 147)	0.042485	0.032227	0.010258
3	(142, 143, 144, 145, 146)	0.042982	0.032837	0.010145
4	(20, 21, 22, 23, 24)	0.043203	0.031982	0.011221
5	(137, 147, 146, 145, 144)	0.043295	0.033569	0.009725

Logical role	Physical qubit
data0	22
anc0	23
data1	24
anc1	25
data2	26

17.3 Budget guard

A guard compares planned QPU time plus a protected reserve with the time remaining (read from the IBM service usage record) before every submission. Remaining time was 590.0 s at 2026-10-26 12:17:27 UTC. Calibration: planned 30.0 s + reserve 130 s against 590.0 s remaining → OK. Part A: planned 20.0 s + reserve 130 s against 588.0 s remaining → OK. The final cleanup cell removes the raw IBM usage snapshot from `backend_record.json`; only the remaining-time figure and its source are kept (Appendix A.2).

18. Circuit design, transpilation and diagrams

Each circuit contains three data qubits, two ancilla qubits, three two-bit syndrome registers (`syn0`, `syn1`, `syn2`) and a three-bit final data register. Each syndrome round resets both ancillas, entangles them with neighbouring data qubits using CNOT gates, then measures them. The suite has two natural configurations (prepared logical 0 and 1) and six single-X injections (data qubit 0, 1 or 2; injected after round 0 or 1). Each configuration exists as a passive and an active circuit: 8 × 2 = 16 circuits.

The **passive** circuit ends after three syndrome rounds and the final data readout. The **active** circuit adds three conditional branches driven by `syn2`: syndrome 01 applies X on data qubit 0, 11 applies X on data qubit 1, and 10 applies X on data qubit 2, followed by the final readout. This is a simple feedback rule that sees only the last syndrome register.

18.1 Circuit metrics (partA_circuit_metrics.csv)

circuit	depth	size	two_qubit_gates	mid_circuit_measure	resets	conditional_blocks	layout	hash
passive / natural_b0	19	27	12	6	6	0	logical	`dd2385ccf9e18d30`
active / natural_b0	22	30	12	6	6	3	logical	`7f998046fb462275`
passive / xq1_after0	20	28	12	6	6	0	logical	`ec01f5c1079cb1ff`
active / xq1_after0	23	31	12	6	6	3	logical	`cae48bed6416db45`
passive / natural_b0 (transpiled)	37	63	12	6	6	0	phys [22, 23, 24, 25, 26]	`746aa09eb258eb67`
active / natural_b0 (transpiled)	40	66	12	6	6	3	phys [22, 23, 24, 25, 26]	`e14b38f918874bfd`
passive / xq1_after0 (transpiled)	38	64	12	6	6	0	phys [22, 23, 24, 25, 26]	`f71a8c9cdcdfa31e`
active / xq1_after0 (transpiled)	41	67	12	6	6	3	phys [22, 23, 24, 25, 26]	`76bf1e9224fbb7aa`

Each circuit has six mid-circuit measurements and six resets. The active versions add three conditional blocks and three levels of depth with no extra two-qubit gates (12 in all cases). Qubits outside the chosen physical line used by any circuit: 0. All 16 circuits and their QPY files were saved before submission (`qpy/partA_logical_circuits.qpy`, `qpy/partA_submitted_isa_circuits.qpy`).

18.2 Circuit diagrams (before = passive, after = active)

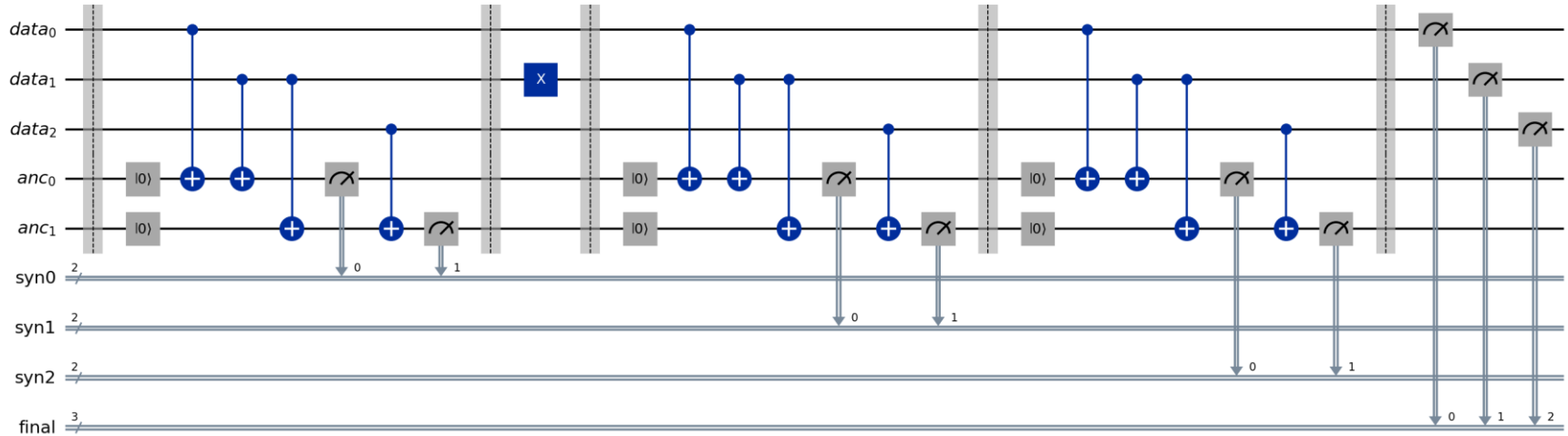


Figure 4. Logical passive (BEFORE) circuit, data qubit 1 fault after round 0 (qec_before_passive_logical.png).

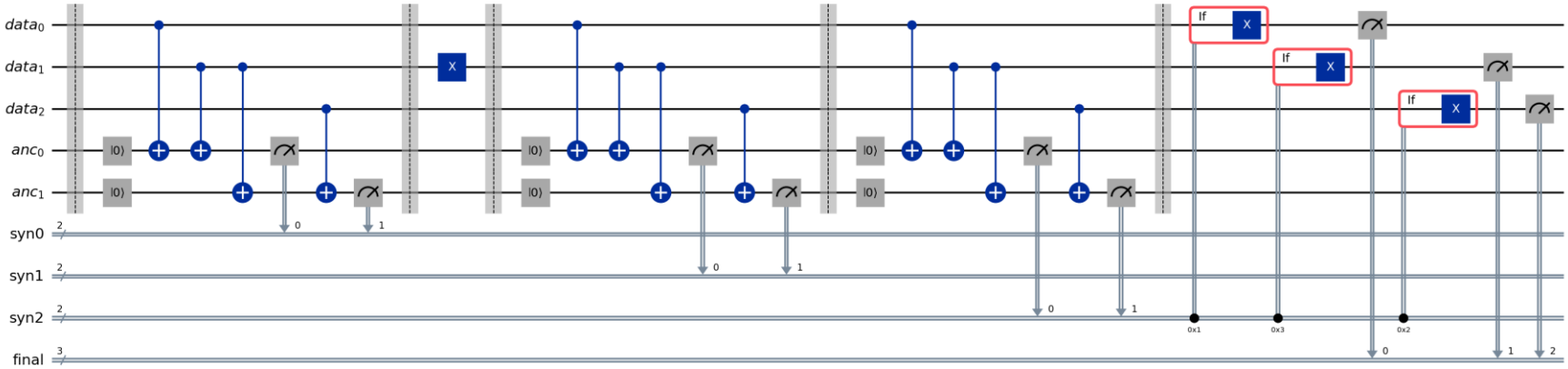


Figure 5. Logical active (AFTER) circuit with conditional X corrections on syn2 (qec_after_active_logical.png).

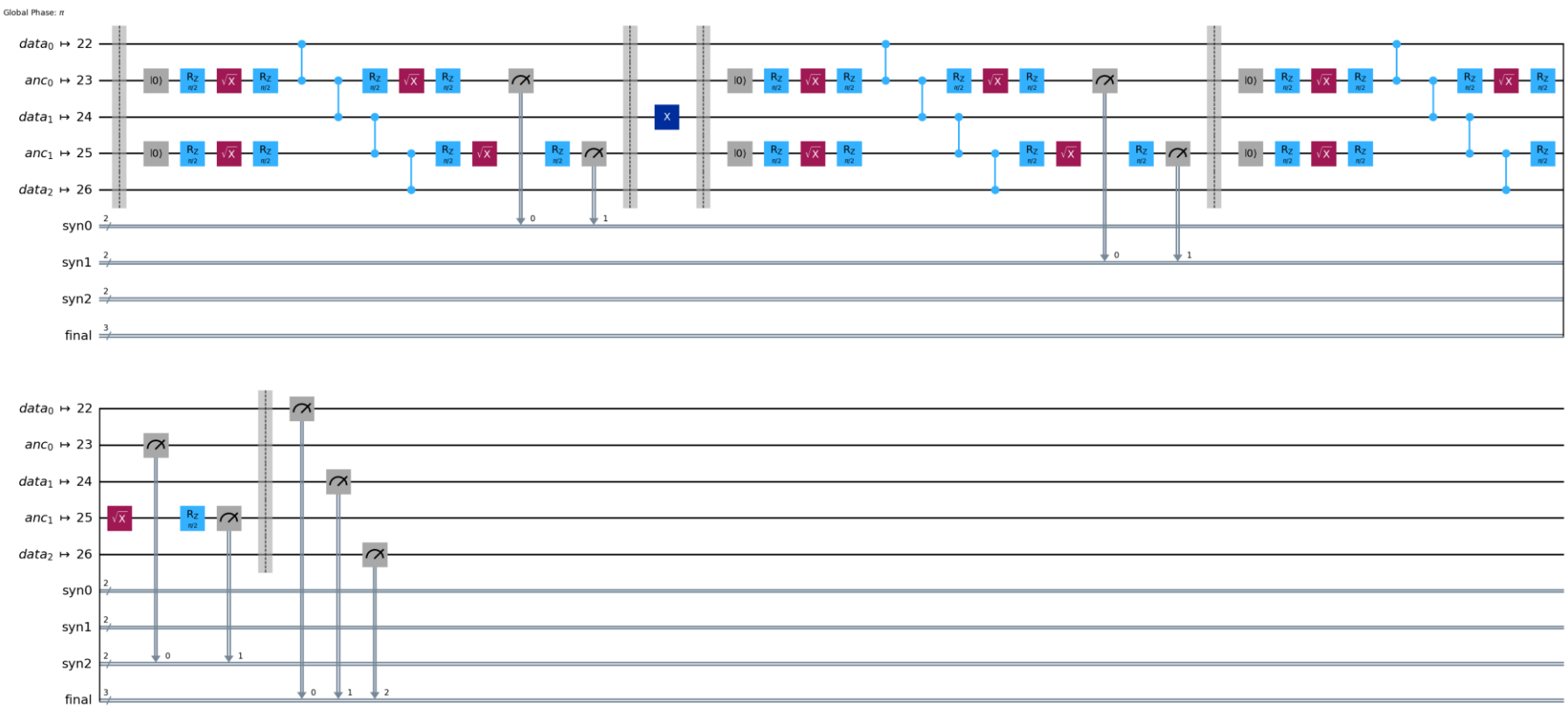


Figure 6. Passive circuit transpiled for ibm_fez onto physical qubits 22–26 (qec_passive_transpiled.png).

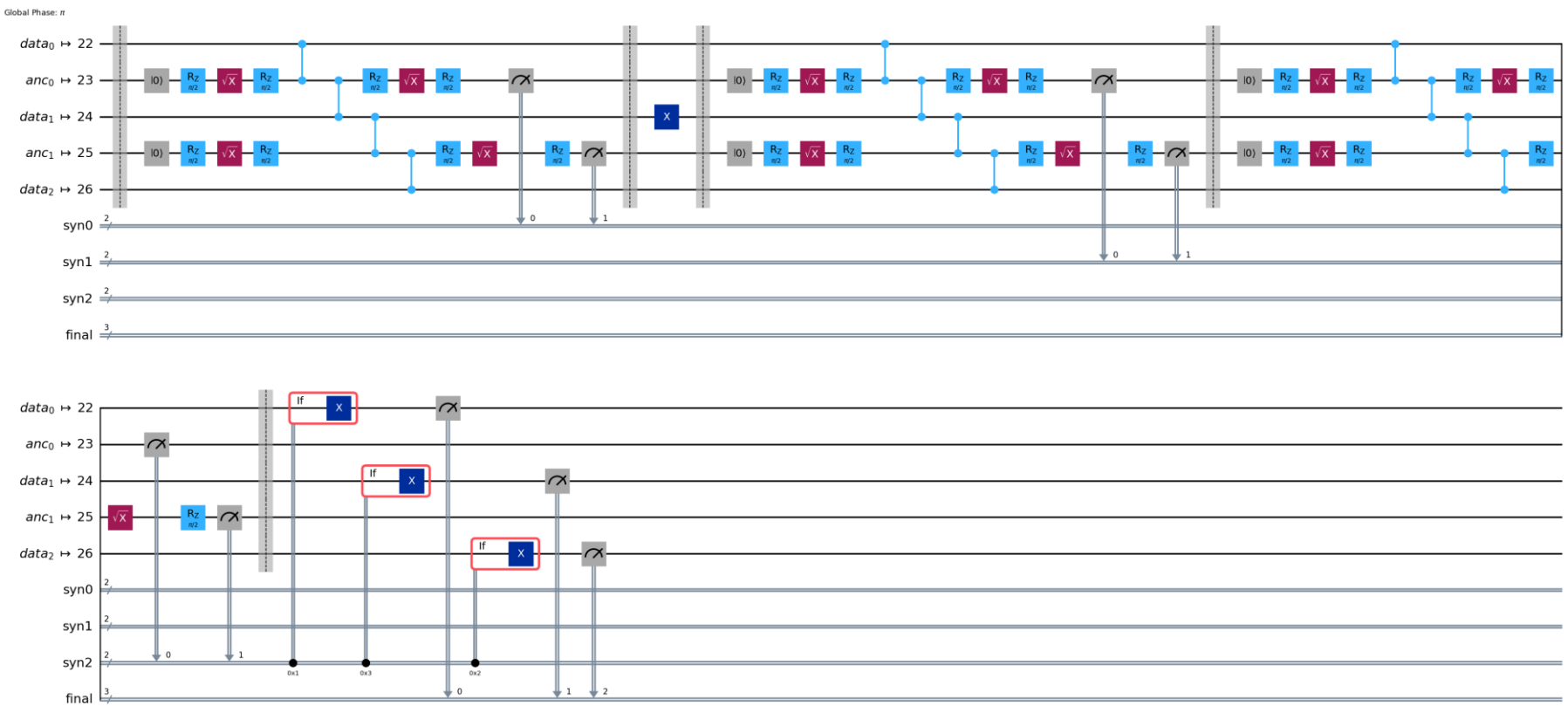


Figure 7. Active circuit transpiled for ibm_fez onto physical qubits 22–26 (qec_active_transpiled.png).

18.3 Submission order (qpy/partA_submitted_order.json)

The 16 circuits were interleaved (passive then active for each configuration) in a single job so that each active circuit is submitted adjacent to its passive counterpart in time.

Position	Configuration	Version	Position	Configuration	Version
0	natural_b0	passive	1	natural_b0	active
2	natural_b1	passive	3	natural_b1	active
4	xq0_after0	passive	5	xq0_after0	active

Position	Configuration	Version	Position	Configuration	Version
6	xq0_after1	passive	7	xq0_after1	active
8	xq1_after0	passive	9	xq1_after0	active
10	xq1_after1	passive	11	xq1_after1	active
12	xq2_after0	passive	13	xq2_after0	active
14	xq2_after1	passive	15	xq2_after1	active

19. Calibration job and Part A hardware results

19.1 Jobs

Stage	Job ID	Backend	Circuits × shots	QPU seconds	Status
Calibration	dbfka8fgdj9s73bbr170	ibm_fez	4 × 100	2.0	DONE
Part A	dbfkb9vpc3tc739nlkeg	ibm_fez	16 × 250	3.0	DONE
Total			20 circuit executions	5.0	

The calibration job returned [100, 100, 100, 100] shots and cost 2.0 QPU seconds, which gave a pessimistic Part A bound of 20 s (suggested cap 40 s). Part A returned 4,000 aligned shots (16 circuits × 250) and used 3.0 QPU seconds. The notebook output states that queue time is not QPU time.

19.2 Per-configuration results (partA_per_configuration.csv)

Configuration	n passive	Passive raw LER	Passive + M0	Passive + M1	Passive + M2	n active	Active before feedback	Active after (on-device)	Commanded share
natural_b0	250	0.0	0.0	0.0	0.0	250	0.004	0.04	0.06
natural_b1	250	0.12	0.032	0.02	0.024	250	0.096	0.064	0.272
xq0_after0	250	0.932	0.012	0.044	0.032	250	0.944	0.1	0.936
xq0_after1	250	0.968	0.008	0.048	0.048	250	0.972	0.08	0.96
xq1_after0	250	0.004	0.024	0.012	0.02	250	0.008	0.044	0.932
xq1_after1	250	0.004	0.036	0.048	0.052	250	0.0	0.048	0.976
xq2_after0	250	0.0	0.012	0.0	0.0	250	0.0	0.004	0.94
xq2_after1	250	0.0	0.02	0.0	0.032	250	0.0	0.004	0.964

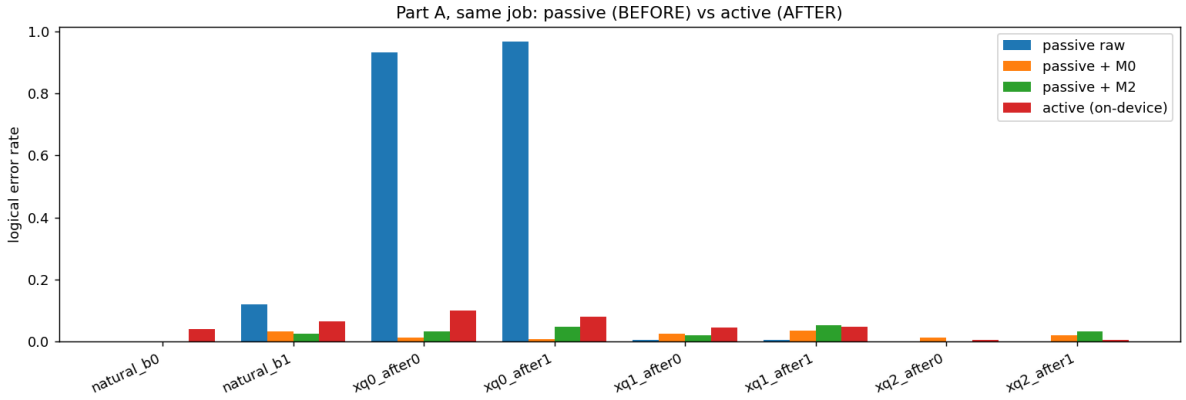


Figure 8. Part A, same job: passive (BEFORE) versus active (AFTER) logical error rate for each of the eight configurations (fig_partA.png).

Raw passive LER is near 0.93 and 0.97 for data qubit 0 faults, because a flip on data qubit 0 is registered as a logical flip, while offline decoding reduces these to 0.008 to 0.048. For the faults on data qubits 1 and 2 the raw LER is already near zero and decoding can only add small errors (up to 0.052 for M2 on xq1_after1). The active circuit brings data-qubit-0 faults from about 0.94–0.97 (before feedback) to 0.08–0.10, but adds roughly 0.04 error on configurations that did not need correction (for example natural_b0: 0.004 → 0.040 and xq1_after*: 0.000–0.008 → 0.044–0.048), which is the cost of correcting on a single noisy syndrome register.

19.3 Pooled results

Condition	Shots	Errors	LER	95% Wilson interval
Passive, raw (no offline decoding)	2,000	507	0.2535	[0.2349, 0.2730]
Passive + M0 offline	2,000	36	0.0180	[0.0130, 0.0248]
Passive + M1 offline	2,000	43	0.0215	[0.0160, 0.0288]
Passive + M2 offline	2,000	52	0.0260	[0.0199, 0.0339]
Active before feedback (counterfactual register view)	2,000	506	0.2530	[0.2344, 0.2725]
Active after on-device correction	2,000	96	0.0480	[0.0395, 0.0583]

Paired contrast	Delta LER	95% interval
Active − passive raw	−0.206	[−0.227, −0.184]
Active − passive + M2	+0.022	[+0.010, +0.034]

The passive raw LER (0.2535) and the active pre-feedback LER (0.2530) agree closely, supporting the interpretation that both circuit versions face the same underlying noise. On-device correction removes about 20.6 percentage points of error but still leaves LER 0.048, whereas offline M0 on passive shots reaches 0.018. A correction was commanded on 1,510 of 2,000 active shots (75.5%).

19.4 Statistics computed from the raw shot file

`partA/partA_raw_shots_dbfkb9vpc3tc739nlkeg.csv` has 4,000 rows with columns `config, version, shot, prepared_bit, syn, final, detectors, observable_error, pre_correction_observable, commanded`. Summaries below were computed directly from it.

Configuration	Version	Shots	Observable error rate	Pre-correction rate	Commanded rate	Mean detector weight
natural_b0	active	250	0.040	0.004	0.060	0.372
natural_b0	passive	250	0.000	0.000	0.056	0.328
natural_b1	active	250	0.064	0.096	0.272	0.696
natural_b1	passive	250	0.120	0.120	0.336	0.884
xq0_after0	active	250	0.100	0.944	0.936	1.272
xq0_after0	passive	250	0.932	0.932	0.924	1.308
xq0_after1	active	250	0.080	0.972	0.960	1.252
xq0_after1	passive	250	0.968	0.968	0.960	1.208
xq1_after0	active	250	0.044	0.008	0.932	2.240
xq1_after0	passive	250	0.004	0.004	0.968	2.244
xq1_after1	active	250	0.048	0.000	0.976	2.176
xq1_after1	passive	250	0.004	0.004	0.980	2.232
xq2_after0	active	250	0.004	0.000	0.940	1.284
xq2_after0	passive	250	0.000	0.000	0.916	1.332
xq2_after1	active	250	0.004	0.000	0.964	1.364
xq2_after1	passive	250	0.000	0.000	0.936	1.300

For passive rows, the pre-correction value equals the observed value and the commanded column records the correction the active rule would command, so only the active rows of the commanded column describe an applied operation.

Detector weight	0	1	2	3	4	5	6
Shots (all 4,000)	736	1,737	1,131	240	132	18	6

Final data register	000	100	001	010	111	110	011	101
Shots (all 4,000)	1,923	524	507	493	385	72	66	30

Most frequent syndrome (syn2 syn1 syn0 order as stored)	000000	000010	000001	001010	000011	000101	001111
Shots	785	482	438	433	432	420	409

20. Frozen decoders on live passive shots

The frozen models and controls were scored on the 2,000 passive shots measured in Part A (status LIVE_OUT_OF_TIME_EVAL). These shots were collected after the freeze, but the set contains a prepared logical 1 and a different fault mix from the internal test, so this is a descriptive drift observation rather than a pure shift experiment. Training pattern coverage is 98.5%, so 1.5% of live shots use unseen patterns.

Method	Live passive LER	95% CI	Delta vs M0	Paired CI for delta	Verdict
No-flip control	0.2535	[0.2340, 0.2735]	+0.2355	[+0.215, +0.256]	worse than M0
Pattern lookup	0.0230	[0.0165, 0.0300]	+0.0050	[−0.002, +0.012]	NOT_STATISTICALLY_RESOLVABLE
M0 MWPM	0.0180	[0.0125, 0.0240]	0.0000	[0.000, 0.000]	reference
M1 MLP	0.0215	[0.0150, 0.0280]	+0.0035	[−0.004, +0.0115]	NOT_STATISTICALLY_RESOLVABLE
M2 MOR router	0.0260	[0.0190, 0.0340]	+0.0080	[0.000, +0.016]	NOT_STATISTICALLY_RESOLVABLE

M0 has the lowest point estimate (0.018), but three of four comparisons with M0 are not statistically resolved. The M2 interval ends exactly at zero (0.000 to +0.016) and is also classified as unresolved.

21. Job recovery, verification and QPU ledger

Both job identifiers were re-queried from IBM after the run. Each is DONE, on a real device, with the correct number of circuits and matching shot counts.

Stage	Job ID	Status	Backend	Real device	Circuits	Shots/circuit	Shots match	QPU s	Verdict
calibration	dbfka8fgdj9s73bbr170	DONE	ibm_fez	True	4	100	True	2.0	OK
partA	dbfkb9vpc3tc739nlkeg	DONE	ibm_fez	True	16	250	True	3.0	OK

21.1 Append-only QPU ledger (qpu_ledger.jsonl)

The ledger has four lines: a submission and a completion record for each job.

UTC time	Job ID	Stage	Event	Backend	Circuits	Shots	QPU s	Status
2026-10-26 12:18:42	dbfka8fgdj9s73bbr170	calibration	submitted	ibm_fez	4	100		
2026-10-26 12:20:27	dbfka8fgdj9s73bbr170	calibration	completed				2	DONE
2026-10-26 12:20:56	dbfkb9vpc3tc739nlkeg	partA	submitted	ibm_fez	16	250		
2026-10-26 12:21:37	dbfkb9vpc3tc739nlkeg	partA	completed				3	DONE

Circuit hashes recorded at submission (first 16 hex digits) allow each submitted circuit to be matched to its QPY file:

Part A circuit	Hash	Part A circuit	Hash
natural_b0|passive	6941628ea05e9249	natural_b0|active	b51e7a2920f43e94
natural_b1|passive	c70fe1a0ba206401	natural_b1|active	4c26f9a67b147998
xq0_after0|passive	a6ce11d55e8b18dd	xq0_after0|active	cef10d4aff2c59e5
xq0_after1|passive	78f648fdef65a00e	xq0_after1|active	af444073e9a1bfb8
xq1_after0|passive	40780ff04c35ef8a	xq1_after0|active	76bf1e9224fbb7aa
xq1_after1|passive	75cb7a4a5fd420f3	xq1_after1|active	ef45612636bed2b8
xq2_after0|passive	694085f1175b1719	xq2_after0|active	291ca3b26ecd908e
xq2_after1|passive	2ba42c22e8a48fb7	xq2_after1|active	184bb34fc631c7fe

Calibration circuit	Hash
natural_b0|passive	7cf5ceb2f02c4f20
natural_b0|active	0271a6712131bdb4
xq1_after0|passive	16e2f1c9bf6108a0
xq1_after0|active	76bf1e9224fbb7aa

22. Single-syndrome demonstration

One detector vector goes in and decisions for every method come out. The interface rejects label dictionaries and Series, wrong width and non-binary input (the rejection tests pass). One worked example per fault class is shown; M0, M1 and M2 agree with the truth on all 8 of these hand-picked rows, which illustrates the interface and is not an evaluation.

Fault class	record_id	Detectors	Truth	M0	M1	M1 prob.	M2	M2 class	M2 decoder
b0 natural	118	00001010	0	0	0	0.122	0	EASY	NEURAL
b0 xq0_after0	1251	00100000	1	1	1	0.999	1	EASY	NEURAL
b0 xq0_after1	3128	00001010	0	0	0	0.122	0	EASY	NEURAL
b0 xq1_after0	1882	00110000	0	0	0	0.002	0	EASY	NEURAL
b0 xq1_after1	3752	00001100	0	0	0	0.053	0	EASY	NEURAL
b0 xq2_after0	2500	00010000	0	0	0	0.010	0	EASY	NEURAL
b0 xq2_after1	4376	00000100	0	0	0	0.000	0	EASY	NEURAL
b1 natural	627	00000011	0	0	0	0.394	0	EASY	NEURAL

Full API call output for the logical-flip example


```
{
 "M0": {"predicted_flip": 1, "score": null, "status": "OK", "latency_us": 109.7},
 "M1": {"predicted_flip": 1, "score": 0.9994653358863874, "status": "OK", "latency_us": 526.5},
 "M2": {"predicted_flip": 1, "score": 0.9961519141834152, "status": "OK",
        "difficulty": "EASY", "decoder": "NEURAL", "latency_us": 1093.3},
 "S1": {"predicted_flip": null, "score": null,
        "status": "NOT_ATTEMPTED (optional stretch, no QPU measurement exists; nothing simulated)"}
}
```


The latencies in this call (109.7, 526.5 and 1,093.3 µs) are first-call values and are larger than the benchmarked medians in Section 23.

23. Latency and computational cost

23.1 Whole-method latency

Method	Batch-1 p50 (µs)	Batch-1 p95 (µs)	Batch per-shot p50 (µs)	Batch per-shot p95 (µs)
Pattern lookup	43.7	88.960	0.330633	0.375990
M0 MWPM	30.3	41.890	0.428967	0.765423
M1 MLP	116.0	482.115	1.020733	1.255767

Method	Batch-1 p50 (µs)	Batch-1 p95 (µs)	Batch per-shot p50 (µs)	Batch per-shot p95 (µs)
M2 MOR router	304.3	543.910	1.264133	1.761537

23.2 M2 stage and class timing (staged probe, batch size 1)

Stage	p50 (µs)	p95 (µs)
S1 difficulty	10.4	19.925
S2 router lookup	0.3	0.605
S3 decoder run	111.6	202.620
S4 output record	0.2	0.300
total	122.3	226.210

Class	Shots timed	p50 (µs)	p95 (µs)
EASY	217	123.1	233.760
HARD	27	127.1	222.770
TRIVIAL	56	11.2	16.175

Batch stage	Per-shot (µs)
S1 difficulty (all shots)	0.039350
S3 MWPM/cache on routed shots	0.013750
S3 neural + Platt on routed shots	0.834283
M2 total	1.264133

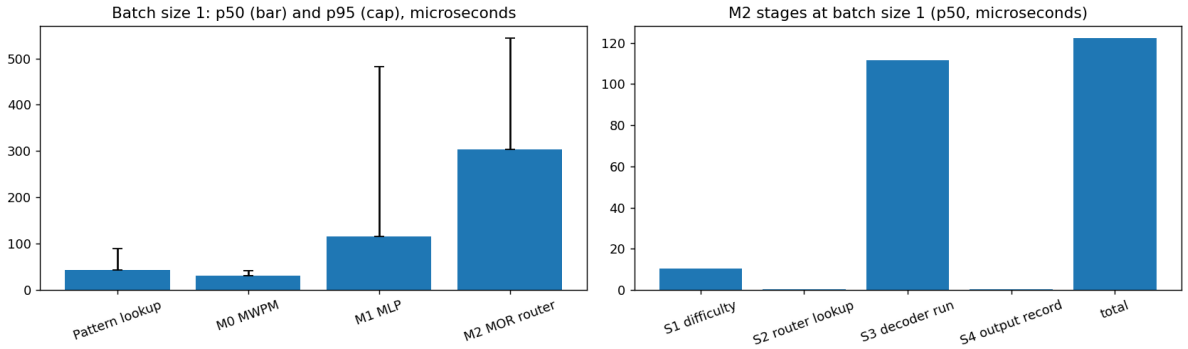


Figure 9. Left: batch-size-1 p50 (bar) and p95 (cap) latency per method. Right: M2 stage latency at batch size 1, in microseconds (fig_latency.png).

M0 is fastest among the required decoders at 30.3 µs p50, followed by M1 (116.0 µs) and M2 (304.3 µs). M2 is slower than M1 at batch size 1 despite skipping the neural call on 20.2% of internal-test shots, because difficulty classification and routing add overhead and most shots still reach the neural network. The staged probe total (122.3 µs) differs from the whole-method p50 (304.3 µs) because it uses a different timing path and sample; the notebook does not reconcile the two, so the staged numbers should be read as a diagnostic, not an additive decomposition. Timings are laptop-class classical Python latency and exclude any QPU round trip.

23.3 QPU cost

Stage	Job ID	Backend	Circuits	Shots	QPU seconds
calibration	`dbfka8fgdj9s73bbr170`	ibm_fez	4	100	2.0
partA	`dbfkb9vpc3tc739nlkeg`	ibm_fez	16	250	3.0

24. Findings, scorecard and end-goal comparison

24.1 Automatically written findings (findings.txt)

The following text is reproduced from `outputs/findings.txt`.

```
FINDINGS
1. Decoding versus no decoding: best LER 0.018 (M1 MLP) against raw LER 0.245; paired delta vs no-flip -0.227 [-
0.253, -0.200] (A better (lower LER)).
2. LER of every method on the internal test: No-flip control 0.245; Pattern lookup 0.020; M0 MWPM 0.028; M1 MLP
0.018; M2 MOR router 0.020.
3. Best vs pattern-lookup control: delta -0.002 [-0.005, +0.000] -> NOT_STATISTICALLY_RESOLVABLE. The comparison
with the strongest control is INCONCLUSIVE; the winner below is exploratory.
4. Tie group by paired interval with M1 MLP: ['M0 MWPM', 'M2 MOR router']; by the latency tie-break the recommended
decoder is M0 MWPM (batch-1 p50 30 us).
5. Router: M2 vs M1 delta +0.002 (NOT_STATISTICALLY_RESOLVABLE); M2 vs M0 delta -0.008
(NOT_STATISTICALLY_RESOLVABLE). It avoided the neural network on 20.2% of internal-test shots, but at batch size 1
it is slower than the plain MLP (304 vs 116 us).
6. Calibration: ECE on test 0.024 (M1 raw), 0.032 (M2 MLP raw), 0.016 (M2 after Platt).
7. Unseen patterns: 5 internal-test shots (0.5%) had patterns not in train; the final test has 0.8% (drift
observation, no labels).
8. Part A (job dbfkb9vpc3tc739nlkeg, ibm_fez): pooled LER passive raw 0.254; passive+M2 0.026; active on-device
0.048; active minus passive raw -0.206 [-0.227, -0.184], active minus passive+M2 +0.022 [+0.010, +0.034].
9. Stretch methods S1 and S2: NOT_ATTEMPTED. No quantum-advantage claim is made or supported.


THREATS TO VALIDITY
- Most labelled shots carry a deliberately injected X, so the fault mix is not natural noise.
- Train and internal test come from one job; the final test is a later job (drift), and has no labels here.
- Only bit-flip errors and a distance-3 code: this is a research prototype, not fault tolerance.
- Only 127 distinct input patterns exist, so a lookup table is nearly as expressive as any model; differences of a
few shots are noise at n=1000.
- Part A uses 250 shots per circuit; per-configuration intervals are wide and the active-vs-passive comparison is
inside one job only.
- Latency is classical Python latency on a laptop, not a real-time control-system claim.
```

24.2 End-goal comparison

Method	LER	Accuracy	Batch-1 p50 (µs)	Neural-call share	Main risk
No-flip control	0.245	0.755	n/a	0.000	decodes nothing (baseline)
Pattern lookup	0.020	0.980	43.7	0.000	unseen patterns default to 0 (0.5% of test)
M0 MWPM	0.028	0.972	30.3	0.000	generic noise model; cannot see flips in some patterns
M1 MLP	0.018	0.982	116.0	1.000	can only learn what training patterns show
M2 MOR router	0.020	0.980	304.3	0.798	routing table is fitted on a small routing slice

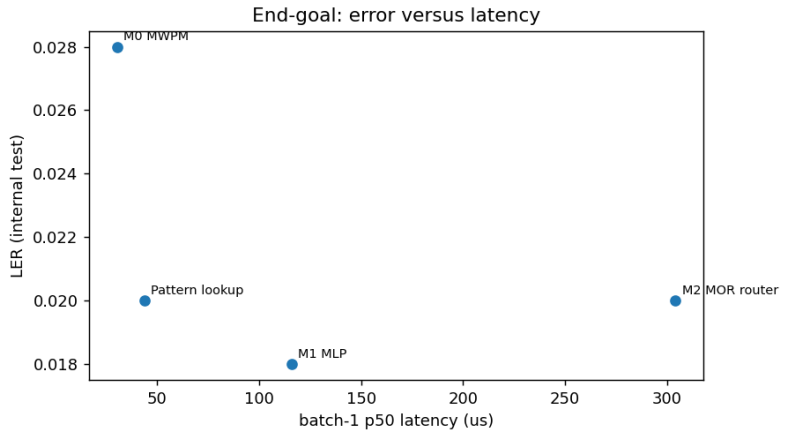


Figure 10. End goal: internal-test error versus batch-1 p50 latency (fig_endgoal.png). M0 is the fastest; M1 has the lowest error; M2 is the slowest with no error gain.

24.3 Scorecard against the judging standards

#	Standard	Evidence	Status
1	Clear problem statement and scope	Section 00 cover	OK

#	Standard	Evidence	Status
2	Clear outcome and deliverables	Sections 12, 19, 21; CSVs in outputs/	OK
3	Leakage-safe data use	Sections 02, 03, 05, 09, 11, 12 (hashes, guards)	OK
4	Methodology comparison	Sections 07 to 10 (identical shots, intervals)	OK
5	Real-hardware circuits BEFORE/AFTER within budget	Sections 13 to 16, 18	OK
6	Technical explanation and insights	Six-field Markdown above every code cell; INSIGHTS lines	CHECK: 6 of 32 code cells lack the six fields
7	Final comparison and end goals	Sections 20 to 22	OK
8	Integrity and reproducibility	Sections 01, 18, 23 (simulator scan, secret scan)	OK
Bonus	Optional quantum stretch (S1, S2)	NOT_ATTEMPTED	NOT_ATTEMPTED

Seven of eight standards pass automatically. The documentation check flags six code cells without the prescribed six-field Markdown header; these cells should be reviewed and the check rerun before submission.

25. Security, simulator and secret audits

25.1 No-simulator scan

The notebook scanned `main.ipynb` against ten forbidden simulator and fake-backend pattern categories. All ten have zero hits and the scan reports PASS.

Forbidden pattern	Hits
local simulator package	0
local simulator symbol	0
local simulator class	0
statevector tools	0
density matrix	0
basic simulator	0
fake backends	0
fake provider	0
generic backend	0
classical eigensolver stand-in	0

25.2 Secret scan

The final cell removed the usage snapshot from `backend_record.json`, cleared saved outputs from notebook files in the submission folder, and scanned for stored IBM token values, CRN-like values and token assignments. Result: API token scan PASS; SECRET SCAN PASS; `.gitignore` ignores `.env` (True); simulator scan on the saved notebook PASS; 34 evidence files indexed. Because the cell clears saved outputs from notebook files in that folder, a working copy with outputs should be kept separately from any sanitised submission copy.

26. Interpretation and discussion

26.1 What the results support

- Decoding is useful on this labelled real-hardware dataset: all four non-trivial approaches cut LER by about 0.22 relative to the no-flip baseline, with paired intervals far from zero.
- M1 has the lowest internal-test point estimate (18 errors in 1,000) with strong precision (0.979) and F1 (0.963), but its difference from the lookup control is not statistically resolved.
- M0 is the fastest single-shot decoder (30.3 µs p50) and is the exploratory recommendation under the declared tie-break; it also has the lowest point estimate on the live passive shots (0.018).
- M2 improves calibration (ECE 0.032 → 0.016) and avoids neural calls on trivial inputs, but is slowest in the tested implementation, so the intended efficiency advantage is not demonstrated.
- On-device conditional correction reduces active-circuit LER from 0.2530 to 0.0480, demonstrating working dynamic-circuit feedback on `ibm_fez`; offline decoding remains better (0.018 to 0.026).
- Two re-queried IBM jobs confirm real-device execution with the requested shot counts; no simulator is used.

26.2 What the results do not support

- A statistically reliable winner among M0, M1, M2 and pattern lookup on the internal test.
- Quantum advantage: all learned and matching decoders run on a classical CPU and the hardware circuits implement a small memory with a simple feedback rule.
- Fault tolerance at scale, performance for arbitrary quantum errors, or generalisation to other code distances and devices.
- A score on the 2,000-row final test, because its labels are unavailable.
- Any statement about the optional S1/S2 quantum-kernel or QAOA+VQC decoders.

26.3 Practical lesson

For a small code with a compact binary feature space, a transparent classical decoder and a strong simple control are hard to beat on practical grounds. The most defensible engineering choice is to keep the low-latency MWPM baseline while retaining the MLP as a promising challenger. More data, more code distances and held-out hardware jobs are needed before the point-estimate ranking can be treated as a firm conclusion.

27. Threats to validity and limitations

Threat	Why it matters	Consequence / mitigation
Injected-fault-heavy data	Most labelled shots carry a deliberately injected X fault, so the label distribution is not natural idle-memory noise.	Results apply to this source dataset and fault mix; collect natural-noise and held-out circuit conditions.
One code distance and one task	All inputs derive from a distance-3 bit-flip repetition code.	Do not generalise to surface codes, higher distance, phase flips or arbitrary Pauli channels.
Small labelled test set	Only 1,000 internal-test shots; the HARD class has 79 shots.	Small LER differences have wide paired uncertainty; add held-out shots and stratified intervals.
Small feature space	Only 127 distinct training patterns; lookup reaches LER 0.020.	MLP gains may reflect a few examples; test on more diverse patterns and codes.

Threat	Why it matters	Consequence / mitigation
Unlabelled final test	2,000 predictions have no observed target.	No final-test accuracy or LER should be claimed.
Distribution and backend shift	Training data come from one `ibm_marrakesh` job; the demonstration ran later on `ibm_fez`.	Out-of-time results mix noise drift, circuit mix, preparation state and backend differences.
Simple active correction	Uses only `syn2`; cannot correct post-syndrome or readout faults reliably.	Treat active-versus-passive as a feedforward demonstration, not an optimal controller.
Timing variation	Python timings depend on OS, CPU, warm-up, batch strategy and cache state.	Compare relative values on one machine; do not interpret as QPU feedback-loop latency.
M2 stage-timing mismatch	Staged total (122.3 µs) differs from whole-method p50 (304.3 µs).	Keep values separate; investigate profiling before deployment claims.
Limited hardware repetitions	250 shots per circuit; active and passive are distinct circuits in one job.	Per-configuration results are noisy; repeat across jobs, calibration windows and shot counts.

28. Conclusion and future work

The project delivers a leakage-aware comparison of MWPM, an MLP and a difficulty-aware router on real IBM repetition-code detector data, together with verified passive and active memory circuits executed on real hardware. All evaluated decoders strongly outperform the no-flip baseline on the internal test. M1 has the lowest point-estimate LER (0.018), but paired intervals do not establish superiority over the lookup control or the other decoders. Under the predeclared tie-break, M0 is the practical exploratory choice because of its 30.3 µs median latency.

The hardware experiment adds an engineering result: active conditional correction lowers measured LER from about 25.3% before feedback to 4.8% after, while offline M0 decoding on passive shots reaches 1.8%. The gap matches the limits of a rule that reacts only to the last syndrome. The strongest next step is to expand the controlled evaluation rather than claim a better decoder from a handful of error-count differences.

28.1 Recommended next steps

- Fix the six code cells that miss the six-field Markdown header, rerun all integrity checks and regenerate the evidence index after final edits.
- Collect larger labelled hardware datasets across several sessions, including more natural-memory shots as well as injected-fault circuits.
- Add code distances or rounds and evaluate on genuinely held-out jobs or backends rather than a random shot split.
- Profile M2 consistently to reconcile staged and whole-method timing, and test routing policies that reduce latency without raising LER.
- Improve active feedback with a decoder that uses the full syndrome history and evaluate it under multiple noise conditions.
- Keep final-test outputs as prediction artefacts until ground-truth labels become available.

Appendix A. Complete JSON output files

The JSON files below are reproduced from `outputs.zip` (arrays on one line for compactness; key order unchanged). SHA-256 hashes in the frozen manifest are full 64-character values.

A.1 outputs/config.json

Frozen experiment parameters.


```
{
  "seed": 2026,
  "boot_single": 1000,
  "boot_paired": 2000,
  "cv_folds": 5,
  "matching_ratio_grid": [0.5, 0.75, 1, 1.25, 1.5, 2, 3],
  "mlp_sizes": [
    [16],
    [32, 32],
    [64, 64],
    [128, 64, 32]
  ],
  "mlp_alphas": [0.0001, 0.01],
  "mlp_max_epochs": 150,
  "mlp_patience": 15,
  "mlp_batch": 64,
  "mlp_lr": 0.001,
  "slices": {
    "fit": 0.6,
    "val": 0.2,
    "route": 0.2
  },
  "difficulty": {
    "trivial": "w == 0",
    "easy": "w in {1, 2}",
    "hard": "w >= 3"
  },
  "shots_partA": 250,
  "shots_calibration": 100,
  "qpu_budget_s": 600,
  "qpu_reserve_s": 130,
  "qpu_partA_cap_s": 40,
  "qpu_calibration_cap_s": 30,
  "transpile_optimization_level": 1,
  "submit_to_qpu": true,
  "backend_name": null,
  "latency_single_shots": 300,
  "latency_batch_repeats": 15
}
```


A.2 outputs/backend_record.json

Backend, qubit line and roles recorded before submission.


```
{
  "backend": "ibm_fez",
  "num_qubits": 156,
  "line": [22, 23, 24, 25, 26],
  "roles": {
    "data0": 22,
    "anc0": 23,
    "data1": 24,
    "anc1": 25,
    "data2": 26
  },
  "remaining_qpu_s": 590,
  "remaining_source": "IBM service.usage()",
  "recorded_utc": "2026-10-26T12:17:27.298930+00:00"
}
```


A.3 outputs/frozen/m0.json

M0 matching weight ratio.


```
{
  "matching_weight_ratio": 0.75,
  "graph": "section 3.7 repetition-code graph"
}
```

A.4 outputs/frozen/manifest.json

Freeze manifest: model hashes, data hashes, configuration, thresholds, routing record and package versions.


```
{
  "frozen_at_utc": "2026-10-26T12:14:35.857901+00:00",
  "seed": 2026,
  "model_files": {
    "m1.joblib": "e9f23ef66d058f0a9f895fdbe22edd980c77f238201700cdd1f957a00d26612f",
    "m2.joblib": "87723f087efe867c3501d9d90db4dbb0f4c76147588fff73cf5254fe50d7b854",
    "lookup.joblib": "94d454b42b8eef79af89cf26d1803aeb4058c7fd335b68d8b121a5520e17ee2e",
    "m0.json": "38def7a5c058d4480d949e31da1bdea243d1f16f8b8ac6a032e8af0d4d060757",
    "m2_slices_record_ids.json": "d7a82a0807dce7591c4cca0c64f713bbe0d1366da1cf05a2f850be6e17461381"
  },
  "data_hashes": {
    "dataset_card.json": "f1f13a49a161e474225846f524e58714401e3d45b039fe2d447bf452c4ca5ff5",
    "final_test_inputs.parquet": "99e219cc34f748434dd1b70cc78c5b4270d1d0d43f6937464644f04328831998",
    "internal_test_1000.parquet": "02c61b3e5d9cd71fd0a6c7f1c15cb022f4325d8b62112e164c37a7606fda2f51",
    "raw/syndromes_d3_r3_v1/ibm_result_verification.json":
"a15866fb0309a216ed9bf9b1eba3792ac6a9e24e3a5367549307d0f3da389577",
    "raw/syndromes_d3_r3_v1/manifest.json": "d0c8052620457305a8fda35ada504feaec516b6410c7921ab210ca067efa6870",
    "raw/syndromes_d3_r3_v1/rep_d3_r3_b0_passive_natural.npz":
"2f58439ff51c82d2e554e32d29b321efaeaeaf06cd4c99239140845ddd14efb3",
    "raw/syndromes_d3_r3_v1/rep_d3_r3_b0_passive_natural.qpy":
"01c18f60551cc852e419596a477c6769ca4e1cf7cb9d5a89fb0fa2b455af8c64",
    "raw/syndromes_d3_r3_v1/rep_d3_r3_b0_passive_xq0_after0.npz":
"ee478715bd54c8e7e4cb0cbf2660d7d84cc041e6cf5dddf959399df81f849b37",
    "raw/syndromes_d3_r3_v1/rep_d3_r3_b0_passive_xq0_after0.qpy":
"df711e5ba62eaa115fcca26e93c884a8395ebdca06347f456bce9fb585429839",
    "raw/syndromes_d3_r3_v1/rep_d3_r3_b0_passive_xq0_after1.npz":
"5bbfcc1b53e90e20b7e0e57010b18b2a0ae6a1ed9acac554e582e0342c2de1c9",
    "raw/syndromes_d3_r3_v1/rep_d3_r3_b0_passive_xq0_after1.qpy":
"dbc2aacf898d8e4daf74bfd72f54eeb7cb347934b0148475e8dedd1465dd7eae",
    "raw/syndromes_d3_r3_v1/rep_d3_r3_b0_passive_xq1_after0.npz":
"8be543d355e7068f800d639dbd7cb0767531337eef643ff161ec6a92117e7ca1",
    "raw/syndromes_d3_r3_v1/rep_d3_r3_b0_passive_xq1_after0.qpy":
"9101cfb3dc777b9967144138a60ae56406b26401215ad888ab13bf501c7bef35",
    "raw/syndromes_d3_r3_v1/rep_d3_r3_b0_passive_xq1_after1.npz":
"f1d455aec9fbeaa39d4974a52ed0b2955b625078b72ccdc61060ff971d354fe4",
    "raw/syndromes_d3_r3_v1/rep_d3_r3_b0_passive_xq1_after1.qpy":
"2057d7d6101ba1553425e9cc9248cb4c1b0b3ada574dc352a28d5243fdf2d806",
    "raw/syndromes_d3_r3_v1/rep_d3_r3_b0_passive_xq2_after0.npz":
"c95a055a0a8a4172c7bc1ffadbfdaf22325a8520363d85ea2dc4d8aebd29b4d9",
    "raw/syndromes_d3_r3_v1/rep_d3_r3_b0_passive_xq2_after0.qpy":
"323816dbd70cef6a1c73bb6f8ca13e644c7c11355084b1afac56448106140afb",
    "raw/syndromes_d3_r3_v1/rep_d3_r3_b0_passive_xq2_after1.npz":
"7bf0668f0a6b7c2bf968c1ffd60848790aacba1be5443e3b068a70a6ead539a8",
    "raw/syndromes_d3_r3_v1/rep_d3_r3_b0_passive_xq2_after1.qpy":
"82b49194aacc36fa1375e6031df345e74497c6e8fd72f9db9782a01a2ec507b3",
    "raw/syndromes_d3_r3_v1/rep_d3_r3_b1_passive_natural.npz":
"515d65d97087d9b925dc44d8d8afd9c117437a38ff34be864f84a96d699566ce",
    "raw/syndromes_d3_r3_v1/rep_d3_r3_b1_passive_natural.qpy":
"d93f42e8b197d9b946b95369f7a650f0f27256bdfa32a009170dc963a6e9c9c4",
    "train_4000.parquet": "203790f8c619e0a09ade1bc7ab35e650119d8224354f6ac30c477d97856d881d"
  },
  "config": {
    "seed": 2026,
    "boot_single": 1000,
    "boot_paired": 2000,
    "cv_folds": 5,
    "matching_ratio_grid": [0.5, 0.75, 1, 1.25, 1.5, 2, 3],
    "mlp_sizes": [
      [16],
      [32, 32],
      [64, 64],
      [128, 64, 32]
    ],
    "mlp_alphas": [0.0001, 0.01],
    "mlp_max_epochs": 150,
    "mlp_patience": 15,
    "mlp_batch": 64,
    "mlp_lr": 0.001,
    "slices": {
      "fit": 0.6,
      "val": 0.2,
      "route": 0.2
    },
    "difficulty": {
      "trivial": "w == 0",
      "easy": "w in {1, 2}",
      "hard": "w >= 3"
    },
    "shots_partA": 250,
    "shots_calibration": 100,
    "qpu_budget_s": 600,
    "qpu_reserve_s": 130,
    "qpu_partA_cap_s": 40,
    "qpu_calibration_cap_s": 30,
    "transpile_optimization_level": 1,
    "submit_to_qpu": true,
    "backend_name": null,
    "latency_single_shots": 300,
    "latency_batch_repeats": 15
  },
  "m0": {
    "ratio": 0.75
  },
  "m1": {
    "config": {
```

```
      "hidden": [64, 64],
      "alpha": 0.01,
      "class_weight": true
    },
    "threshold": 0.8400000000000001,
    "epochs": 58,
    "n_params": 4801
  },
  "m2": {
    "routing": {
      "TRIVIAL": "IDENTITY",
      "EASY": "NEURAL",
      "HARD": "NEURAL"
    },
    "difficulty_thresholds": {
      "trivial": "w == 0",
      "easy": "w in {1, 2}",
      "hard": "w >= 3"
    },
    "mwpm_ratio": 0.75,
    "platt_a_b": [0.8384677387653253, -0.9933551275259521],
    "neural_threshold": 0.5,
    "slice_sizes": {
      "fit_idx": 2400,
      "val_idx": 800,
      "route_idx": 800
    }
  },
  "versions": {
    "qiskit": "2.5.0",
    "qiskit_ibm_runtime": "0.48.0",
    "pymatching": "2.4.0",
    "pandas": "2.3.2",
    "numpy": "2.1.0",
    "scipy": "1.18.0",
    "sklearn": "1.7.1",
    "matplotlib": "3.10.6",
    "joblib": "1.5.3"
  }
}
```


A.5 outputs/final_predictions_manifest.json

Hashes of prediction CSVs and the freeze timestamp they followed.


```
{
  "written_after_freeze": "2026-10-26T12:14:35.857901+00:00",
  "files": {
    "predictions_final_test_M0.csv": "618122df07f32d2bb5e88c6aef9036766222f586d48de6b55c1c7503484a55ae",
    "predictions_final_test_M1.csv": "ffaf811b61fcb1d219e3bf877d78665a241ed80041d5fe9cab7dbe390e490c99",
    "predictions_final_test_M2.csv": "55d7b0d1b46cbf41191190d4129719665b34a36d0d7661a03ed88e0483264f3b",
    "predictions_internal_test_M0.csv": "0354bbe357fd65e8ca3dab44cbe27239c2060281ab2ef20a7821c66c0bf2da08",
    "predictions_internal_test_M1.csv": "c30a3c47589fc654e9a61495613f8b90714578cf041bea78806d072f1a12f3cb",
    "predictions_internal_test_M2.csv": "5f0172a77cf4d6d2e47ccdb0fa877929d379216bc07ed780958f20ec56b16ecf"
  }
}
```


A.6 outputs/qpy/partA_submitted_order.json

Order in which the 16 circuits were submitted.


```
[
  ["natural_b0", "passive"],
  ["natural_b0", "active"],
  ["natural_b1", "passive"],
  ["natural_b1", "active"],
  ["xq0_after0", "passive"],
  ["xq0_after0", "active"],
  ["xq0_after1", "passive"],
  ["xq0_after1", "active"],
  ["xq1_after0", "passive"],
  ["xq1_after0", "active"],
  ["xq1_after1", "passive"],
  ["xq1_after1", "active"],
  ["xq2_after0", "passive"],
  ["xq2_after0", "active"],
  ["xq2_after1", "passive"],
  ["xq2_after1", "active"]
]
```


A.7 outputs/qpu_ledger.jsonl

Append-only ledger; each line is one JSON record.


```
{"ts_utc":"2026-10-26T12:18:42.430631+00:00","job_id":"dbfka8fgdj9s73bbr170","stage":"calibration","backend":"
ibm_fez","shots":100,"n_circuits":4,"circuit_names":["natural_b0|passive","natural_b0|active","xq1_after0|pass
ive","xq1_after0|active"],"circuit_hashes":["7cf5ceb2f02c4f20","0271a6712131bdb4","16e2f1c9bf6108a0","76bf1e92
24fbb7aa"],"layout":{"data0":22,"anc0":23,"data1":24,"anc1":25,"data2":26},"qpu_seconds":null,"status_final":n
ull,"qpy_file":"qpy/partA_submitted_isa_circuits.qpy"}
{"ts_utc":"2026-10-26T12:20:27.227457+00:00","job_id":"dbfka8fgdj9s73bbr170","stage":"calibration","qpu_second
```

```
s":2,"status_final":"DONE"}
{"ts_utc":"2026-10-26T12:20:56.872117+00:00","job_id":"dbfkb9vpc3tc739nlkeg","stage":"partA","backend":"ibm_fe
z","shots":250,"n_circuits":16,"circuit_names":["natural_b0|passive","natural_b0|active","natural_b1|passive",
"natural_b1|active","xq0_after0|passive","xq0_after0|active","xq0_after1|passive","xq0_after1|active","xq1_aft
er0|passive","xq1_after0|active","xq1_after1|passive","xq1_after1|active","xq2_after0|passive","xq2_after0|act
ive","xq2_after1|passive","xq2_after1|active"],"circuit_hashes":["6941628ea05e9249","b51e7a2920f43e94","c70fe1
a0ba206401","4c26f9a67b147998","a6ce11d55e8b18dd","cef10d4aff2c59e5","78f648fdef65a00e","af444073e9a1bfb8","40
780ff04c35ef8a","76bf1e9224fbb7aa","75cb7a4a5fd420f3","ef45612636bed2b8","694085f1175b1719","291ca3b26ecd908e"
,"2ba42c22e8a48fb7","184bb34fc631c7fe"],"layout":{"data0":22,"anc0":23,"data1":24,"anc1":25,"data2":26},"qpu_s
econds":null,"status_final":null,"qpy_file":"qpy/partA_submitted_isa_circuits.qpy"}
{"ts_utc":"2026-10-26T12:21:37.815293+00:00","job_id":"dbfkb9vpc3tc739nlkeg","stage":"partA","qpu_seconds":3,"
status_final":"DONE"}
```


A.8 outputs/evidence_index.json

Index of evidence files with sizes and truncated SHA-256 values (also tabulated in Appendix C).


```
{
  "generated_utc": "2026-10-26T12:27:23.444735+00:00",
  "job_ids": {
    "calibration": "dbfka8fgdj9s73bbr170",
    "partA": "dbfkb9vpc3tc739nlkeg"
  },
  "files": {
    "backend_record.json": {
      "bytes": 350,
      "sha256": "7f608e9f4e74e4e3"
    },
    "config.json": {
      "bytes": 982,
      "sha256": "d0eb92726ebf4260"
    },
    "evidence_index.json": {
      "bytes": 3748,
      "sha256": "390618e28108c4ea"
    },
    "figures\\fig_benchmark.png": {
      "bytes": 87018,
      "sha256": "03c16d89ce2fd40e"
    },
    "figures\\fig_eda.png": {
      "bytes": 86882,
      "sha256": "81407f1aa5569ce0"
    },
    "figures\\fig_endgoal.png": {
      "bytes": 28992,
      "sha256": "ecfdc1e131f1523e"
    },
    "figures\\fig_latency.png": {
      "bytes": 53930,
      "sha256": "18e5b2c43695a3f7"
    },
    "figures\\fig_partA.png": {
      "bytes": 42614,
      "sha256": "8a479d9e310b0e0e"
    },
    "figures\\fig_reliability.png": {
      "bytes": 73729,
      "sha256": "d8f2e4d46a3dfa0e"
    },
    "figures\\qec_active_transpiled.png": {
      "bytes": 136855,
      "sha256": "eaf6e38e3ab3a735"
    },
    "figures\\qec_after_active_logical.png": {
      "bytes": 82423,
      "sha256": "97bdedcc67687766"
    },
    "figures\\qec_before_passive_logical.png": {
      "bytes": 71782,
      "sha256": "51643f2385542e8b"
    },
    "figures\\qec_passive_transpiled.png": {
      "bytes": 127688,
      "sha256": "7632ec6b4da29d92"
    },
    "final_predictions_manifest.json": {
      "bytes": 730,
      "sha256": "198d07b402803e70"
    },
    "findings.txt": {
      "bytes": 2135,
      "sha256": "8d46955e322e6afb"
    },
    "frozen\\lookup.joblib": {
      "bytes": 568,
      "sha256": "94d454b42b8eef79"
    },
    "frozen\\m0.json": {
      "bytes": 86,
      "sha256": "38def7a5c058d448"
    },
    "frozen\\m1.joblib": {
      "bytes": 162812,
      "sha256": "e9f23ef66d058f0a"
    },
    "frozen\\m2.joblib": {
      "bytes": 163444,
      "sha256": "87723f087efe867c"
    },
```

```
    "frozen\\m2_slices_record_ids.json": {
      "bytes": 23148,
      "sha256": "d7a82a0807dce759"
    },
    "frozen\\manifest.json": {
      "bytes": 5478,
      "sha256": "10a26c99e4ded457"
    },
    "partA\\partA_raw_shots_dbfkb9vpc3tc739nlkeg.csv": {
      "bytes": 596347,
      "sha256": "ebb3b9fb857a0c13"
    },
    "partA_circuit_metrics.csv": {
      "bytes": 718,
      "sha256": "ca9e3da074b83b53"
    },
    "partA_per_configuration.csv": {
      "bytes": 603,
      "sha256": "5ba8de6daaa727e7"
    },
    "predictions_final_test_M0.csv": {
      "bytes": 16923,
      "sha256": "618122df07f32d2b"
    },
    "predictions_final_test_M1.csv": {
      "bytes": 56416,
      "sha256": "ffaf811b61fcb1d2"
    },
    "predictions_final_test_M2.csv": {
      "bytes": 47987,
      "sha256": "55d7b0d1b46cbf41"
    },
    "predictions_internal_test_M0.csv": {
      "bytes": 8816,
      "sha256": "0354bbe357fd65e8"
    },
    "predictions_internal_test_M1.csv": {
      "bytes": 28556,
      "sha256": "c30a3c47589fc654"
    },
    "predictions_internal_test_M2.csv": {
      "bytes": 24299,
      "sha256": "5f0172a77cf4d6d2"
    },
    "qpu_ledger.jsonl": {
      "bytes": 1855,
      "sha256": "ce36c52e81209206"
    },
    "qpy\\partA_logical_circuits.qpy": {
      "bytes": 37846,
      "sha256": "db8891668ab18082"
    },
    "qpy\\partA_submitted_isa_circuits.qpy": {
      "bytes": 156582,
      "sha256": "77a74d24354420e2"
    },
    "qpy\\partA_submitted_order.json": {
      "bytes": 424,
      "sha256": "3566d897a1f15cab"
    }
  },
  "qpu_seconds_total": 5,
  "status": {
    "hardware": "OK",
    "partA": "OK",
    "stretch": "NOT_ATTEMPTED"
  }
}
```

Appendix B. Complete CSV analysis outputs and data samples

B.1 partA_per_configuration.csv (complete)

config	n_passive	passive_raw_LER	passive+M0_LER	passive+M1_LER	passive+M2_LER	n_active	active_before_feedback_LER	active_LER	commanded_share
natural_b0	250	0.0	0.0	0.0	0.0	250	0.004	0.04	0.06
natural_b1	250	0.12	0.032	0.02	0.024	250	0.096	0.064	0.272
xq0_after0	250	0.932	0.012	0.044	0.032	250	0.944	0.1	0.936
xq0_after1	250	0.968	0.008	0.048	0.048	250	0.972	0.08	0.96
xq1_after0	250	0.004	0.024	0.012	0.02	250	0.008	0.044	0.932
xq1_after1	250	0.004	0.036	0.048	0.052	250	0.0	0.048	0.976
xq2_after0	250	0.0	0.012	0.0	0.0	250	0.0	0.004	0.94
xq2_after1	250	0.0	0.02	0.0	0.032	250	0.0	0.004	0.964

B.2 partA_circuit_metrics.csv (complete)

circuit	depth	size	two_qubit_gates	mid_circuit_measure	resets	conditional_blocks	layout	hash
passive / natural_b0	19	27	12	6	6	0	logical	dd2385ccf9e18d30
active / natural_b0	22	30	12	6	6	3	logical	7f998046fb462275
passive / xq1_after0	20	28	12	6	6	0	logical	ec01f5c1079cb1ff
active / xq1_after0	23	31	12	6	6	3	logical	cae48bed6416db45
passive / natural_b0 (transpiled)	37	63	12	6	6	0	phys [22, 23, 24, 25, 26]	746aa09eb258eb67
active / natural_b0 (transpiled)	40	66	12	6	6	3	phys [22, 23, 24, 25, 26]	e14b38f918874bfd
passive / xq1_after0 (transpiled)	38	64	12	6	6	0	phys [22, 23, 24, 25, 26]	f71a8c9cdcdfa31e
active / xq1_after0 (transpiled)	41	67	12	6	6	3	phys [22, 23, 24, 25, 26]	76bf1e9224fbb7aa

B.3 partA/partA_raw_shots_dbfkb9vpc3tc739nlkeg.csv (4,000 rows; sample of 6 rows per version for config xq0_after0)

The full file holds one row per shot. The detector column is stored as a Python tuple of numpy integers in the CSV; below it is shown as a plain bit string. The row-level file is included in the supplied `outputs.zip`; the aggregate statistics for all 4,000 rows are in Section 19.4.

config	version	shot	prepared_bit	syn	final	detectors	observable_error	pre_correction	commanded
xq0_after0	passive	0	0	001010	100	00100000	1	1	1
xq0_after0	passive	1	0	001010	100	00100000	1	1	1
xq0_after0	passive	2	0	001010	100	00100000	1	1	1
xq0_after0	passive	3	0	011111	100	01100001	1	1	1
xq0_after0	passive	4	0	001010	100	00100000	1	1	1
xq0_after0	passive	5	0	001010	100	00100000	1	1	1
xq0_after0	active	0	0	001010	000	00100000	0	1	1
xq0_after0	active	1	0	001010	000	00100000	0	1	1
xq0_after0	active	2	0	001010	000	00100000	0	1	1
xq0_after0	active	3	0	011010	000	01110000	0	1	1
xq0_after0	active	4	0	000010	000	00001000	0	1	1
xq0_after0	active	5	0	001010	000	00100000	0	1	1

B.4 predictions_internal_test_M0/M1/M2.csv (1,000 rows each; first 8 rows)

record_id	M0 flip	M0 p_flip	M1 flip	M1 p_flip	M2 flip	M2 p_flip
3440	1	(blank)	1	0.999465	1	0.996152
4790	0	(blank)	0	0.000416	0	0.000372
4864	0	(blank)	0	0.000416	0	0.000372
1179	0	(blank)	0	0.001269	0	(blank)
1166	0	(blank)	0	0.141236	0	0.057396
2269	0	(blank)	0	0.001897	0	0.004912
1515	1	(blank)	1	0.999150	1	0.990538
2927	0	(blank)	0	0.010451	0	0.000669

The three internal-test files list the same 1,000 record IDs in the same (shuffled) order, so each row compares the three decoders on the same shot.

B.5 predictions_final_test_M0/M1/M2.csv

The three 2,000-row final-test files have columns `record_id, predicted_flip, p_flip`, are validated (OK) and are summarised in Section 16.2 and 16.3 with the first eight rows. Their SHA-256 hashes are in Appendix A.5.

Appendix C. Evidence index (34 files)

Generated 2026-10-26T12:27:23.444735+00:00. Job IDs: calibration dbfka8fgdj9s73bbr170; Part A dbfkb9vpc3tc739nlkeg. Total QPU seconds recorded in the index: 5. Index status: [object Object].

#	File (as stored)	Bytes	SHA-256 (first 16)
1	backend_record.json	350	7f608e9f4e74e4e3
2	config.json	982	d0eb92726ebf4260
3	evidence_index.json	3748	390618e28108c4ea
4	figures\fig_benchmark.png	87018	03c16d89ce2fd40e
5	figures\fig_eda.png	86882	81407f1aa5569ce0
6	figures\fig_endgoal.png	28992	ecfdc1e131f1523e
7	figures\fig_latency.png	53930	18e5b2c43695a3f7
8	figures\fig_partA.png	42614	8a479d9e310b0e0e
9	figures\fig_reliability.png	73729	d8f2e4d46a3dfa0e
10	figures\qec_active_transpiled.png	136855	eaf6e38e3ab3a735
11	figures\qec_after_active_logical.png	82423	97bdedcc67687766
12	figures\qec_before_passive_logical.png	71782	51643f2385542e8b
13	figures\qec_passive_transpiled.png	127688	7632ec6b4da29d92
14	final_predictions_manifest.json	730	198d07b402803e70
15	findings.txt	2135	8d46955e322e6afb
16	frozen\lookup.joblib	568	94d454b42b8eef79
17	frozen\m0.json	86	38def7a5c058d448
18	frozen\m1.joblib	162812	e9f23ef66d058f0a
19	frozen\m2.joblib	163444	87723f087efe867c
20	frozen\m2_slices_record_ids.json	23148	d7a82a0807dce759
21	frozen\manifest.json	5478	10a26c99e4ded457
22	partA\partA_raw_shots_dbfkb9vpc3tc739nlkeg.csv	596347	ebb3b9fb857a0c13
23	partA_circuit_metrics.csv	718	ca9e3da074b83b53
24	partA_per_configuration.csv	603	5ba8de6daaa727e7
25	predictions_final_test_M0.csv	16923	618122df07f32d2b

#	File (as stored)	Bytes	SHA-256 (first 16)
26	predictions_final_test_M1.csv	56416	ffaf811b61fcb1d2
27	predictions_final_test_M2.csv	47987	55d7b0d1b46cbf41
28	predictions_internal_test_M0.csv	8816	0354bbe357fd65e8
29	predictions_internal_test_M1.csv	28556	c30a3c47589fc654
30	predictions_internal_test_M2.csv	24299	5f0172a77cf4d6d2
31	qpu_ledger.jsonl	1855	ce36c52e81209206
32	qpy\partA_logical_circuits.qpy	37846	db8891668ab18082
33	qpy\partA_submitted_isa_circuits.qpy	156582	77a74d24354420e2
34	qpy\partA_submitted_order.json	424	3566d897a1f15cab

Appendix D. Equations and definitions

Symbol / term	Definition used in this report
x_i	Eight-bit detector vector for shot i.
y_i	Ground-truth logical observable error; 1 when the defined logical observable flipped.
ŷ_i	Decoder prediction of the logical-flip label.
LER	(1/N) Σ 1[ŷ_i ≠ y_i], the fraction of shots decoded incorrectly.
Accuracy	1 − LER for the binary decision task.
Precision	TP / (TP + FP) for the logical-flip class.
Recall	TP / (TP + FN) for the logical-flip class.
F1	2 × Precision × Recall / (Precision + Recall).
Balanced accuracy	Mean of recall on the flip class and recall on the no-flip class.
Detector weight w	Sum of the eight detector bits; defines TRIVIAL (w = 0), EASY (w = 1 or 2) and HARD (w ≥ 3).
Paired delta	LER(A) − LER(B) on the same shots or resamples.
ECE	Expected calibration error: weighted gap between mean predicted probability and observed event fraction over probability bins.
Platt scaling	Calibrated probability = sigmoid(a · s + b) for a raw score s; here a = 0.8385, b = −0.9934.
Wilson interval	Confidence interval for a binomial proportion used for hardware LER.
M0	PyMatching MWPM decoder with training-selected measurement/data edge-weight ratio 0.75.
M1	Classical MLP (64, 64) with validation-tuned threshold 0.84.
M2	Difficulty-aware router with identity/neural paths, Platt calibration and a cached matching function.

Appendix E. References

[1] Kaustubh Ladda, `main.ipynb`, "Q-DecodeQEC: decoding real IBM repetition-code syndromes (M0 matching, M1 MLP, M2 MOR router) and a passive vs active memory circuit on real hardware." Submitted notebook, its execution outputs and `outputs.zip`; evidence generated 26 October 2026. Primary source for all project-specific numbers in this report.

[2] O. Higgott, "PyMatching: A Python Package for Decoding Quantum Codes with Minimum-Weight Perfect Matching," ACM Transactions on Quantum Computing, vol. 3, no. 3, Article 16, 2022. DOI: 10.1145/3505637.

[3] IBM Quantum Documentation, "Execute dynamic circuits" (mid-circuit measurements and classical control flow). https://quantum.cloud.ibm.com/docs/en/guides/execute-dynamic-circuits

[4] IBM Quantum Documentation, "SamplerV2," Qiskit Runtime API reference. https://quantum.cloud.ibm.com/docs/api/qiskit-ibm-runtime/sampler-v2

[5] Trinexis × IEEE, IBM Qiskit Fall Fest 2026, Q-DecodeQEC challenge repository and contribution instructions. https://github.com/Trinexis-Qiskit/Quantum-QEC